NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA SURATHKAL

DEPARTMENT OF INFORMATION TECHNOLOGY

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE

FUNDAMENTALS OF COMPUTER SYSTEMS

IT111

SEMESTER I

ANSWER KEY END SEMESTER EXAM

PREPARED BY

Dr. SURESAN PARETH

Dr.Suresh Pareth

Contents

1 FUNDAMENTALS OF COMPUTER SYSTEMS **1**

2 Course Structure **2**

3 Evaluation Criteria **3**

4 Y86-64 Architecture **4**
4.1 Key Components of the Y86-64 CPU 4

5 Example: Running a Simple Program **16**

6 Role of the Header in Executable and Execution of Subtraction in Y86-64 **21**
6.1 C Code for Subtraction 22

7 Role of the OS and Execution of Subtraction in Y86-64 **25**

8 Instruction Execution in SEQ **28**

9 Pipeline **30**

10 Pipeline Hazards and Forwarding in Y86-64 **34**
10.1 Pipeline Stages Without Forwarding 35

11 Overview of Processor Architecture (Questions and Answers) **37**

12 Section 4.5: Pipelining **43**

13 Performance Optimization **47**
13.1 Questions and Answers on Y86-64 Pipelining and Performance Optimization 47

14 Programming and Problem-Solving **52**

15 Blooms's Level 5: Evaluating Questions **57**

16 Bloom's Level 6: Creating Questions **62**

17 Linking **66**

Dr.Sureshparet

18 Example: Linking Two Separate Source Files **72**

19 Memory Hierarchy: Questions and Answers **76**

20 Exceptional Control Flow **81**
20.1 Questions and Answers on Normal Control Flow 81
20.2 Questions and Answers on Exceptions in Control Flow 84

21 Virtual Memory **86**
21.1 Questions and Answers on Virtual Memory 86

22 Operating System **88**
22.1 Introduction 88

23 WEB TECHNOLOGIES AND APPLICATIONS **90**
23.1 Questions and Answers: 90
23.2 Level 3: Apply 90
23.3 Level 4: Analyze 92
23.4 Level 5: Evaluate 93
23.5 Level 6: Create 94

24 FUNDAMENTALS OF COMPUTER SYSTEMS **111**

25 Evaluation Criteria **112**

26 END SEMESTER EXAMINATION ANSWER KEY **113**

Dr.Sureshpareth

1

FUNDAMENTALS OF COMPUTER SYSTEMS

Sub Code Evaluation Criteria	IT111 QUIZ(2) + MIDTERM + ENDTERM
L-T-P	4-0-0
Total Hours Total Marks	45(T) 100
Credits Exam Hours	04 03

Dr.Suresh

2

Course Structure

Text Books:

1.Randal E. Bryant, David R. O'Hallaron, Computer Systems: A Programmer's Perspective, Pearson, 2016.

2.Carl HAmacher, Zvonko Vranesic, SAfeat Zaky, Computer Organization, McGraw Hill, 2011'

3.Abraham Silberschatz, PEter B Galvin, Gerrg Gagne, Operating Systems, Wiley, 2015.

Dr.Sureshpare

3 Evaluation Criteria

Quiss(2) (20%);

Mid Term(30%);

Final Exam (50%).

Week	Content
Quiz	Covers Weeks 1-4
Midterm Examination	Covers Weeks 1-7
Endterm Examination	Covers Weeks 1-15

Dr.Suresh Pare

4 Y86-64 Architecture

The Y86-64 architecture is a simplified, instructional version of the x86-64 architecture. It is designed to teach the principles of modern computer systems without the complexity of the full x86-64 architecture. Below is a detailed explanation of its key topics:

4.1 Key Components of the Y86-64 CPU

What is a Processor?

A processor, also known as the Central Processing Unit (CPU), is often referred to as the "brain" of the computer. Its main job is to carry out instructions in a computer program, processing data and sending outputs based on those instructions. At its core, a processor executes a set of simple, basic operations (like arithmetic calculations or data movement) at incredible speeds to complete complex tasks. It interacts with memory, where instructions and data are stored, and with input/output (I/O) devices, allowing the computer to connect with other devices, networks, and users.

The processor is central to the computer's operation because it performs the core tasks needed to run software, from system-level programs like operating systems to application-level programs like browsers or games.

Functions of the Processor::

The main functions of the processor can be summarized as follows:

Fetch: Retrieve the next instruction from memory.

Decode: Interpret the instruction and determine what actions to perform.

Execute: Carry out the specified operation, which may involve calculations, data movement, or interacting with other components.

Store: Write the result to a register or memory if needed.

This cycle repeats rapidly, enabling the processor to handle millions or even billions of instructions per second.

Arithmetic Logic Unit (ALU): Responsible for performing arithmetic and logic operations.

Dr.Sureshpare

The Arithmetic Logic Unit (ALU) is a crucial component of the processor that performs all arithmetic (like addition, subtraction) and logic operations (like comparisons and logical AND, OR operations). It's essentially the "calculator" within the CPU, responsible for handling:

Arithmetic Operations: These include addition, subtraction, multiplication, and division. Although complex arithmetic (like multiplication) might seem basic, these operations are fundamental to everything from calculations in spreadsheets to 3D graphics in games.

Logic Operations: These involve Boolean operations, such as AND, OR, NOT, and XOR, which are essential for decision-making in programs. For example, checking if a number is positive or comparing two values are operations managed by the ALU.

The ALU receives data from registers (storage areas within the CPU), performs operations, and then stores the results back in registers or memory. It's optimized to perform these operations very quickly, often completing multiple simple operations in a single cycle

Registers: Small, fast storage locations within the processor that hold data temporarily.

Control Logic: Directs how instructions are executed, managing the overall flow within the processor.

The CPU architecture comprises several functional units that together implement the fetch-decode-execute cycle. These units work together to process instructions in a pipeline for improved efficiency.

1. Registers:

Registers are small, very fast storage locations inside the processor. Think of them as "scratchpads" or "short-term memory" for the CPU, holding data and instructions that the processor is currently working on. Unlike the main memory (RAM), registers are directly integrated into the processor and are much faster to access, enabling the CPU to avoid delays when executing instructions.

There are different types of registers, each with a specific role:

General-purpose Registers: Store intermediate data and results. They are flexible and can be used for various purposes in different programs.

Special-purpose Registers: These serve specific functions, such as:

Program Counter (PC): Holds the address of the next instruction to execute.

Instruction Register (IR): Stores the instruction currently being executed.

Accumulator: Temporarily stores intermediate results from the ALU.

By using registers, the CPU minimizes the time spent accessing slower memory (RAM), allowing it to work efficiently on active data and instructions.

2. Control Logic:

The Control Logic component is like the "conductor" of an orchestra—it coordinates all activities within the processor to ensure instructions are carried out in the correct order and manner. Control logic manages:

Instruction Decoding: It interprets each instruction fetched from memory to determine what action is needed.

Data Routing: It directs data to and from the ALU, registers, and memory based on the decoded instructions.

Timing and Synchronization: The control unit orchestrates when and how each part of the CPU operates, often governed by a system clock.

Dr. Suresh Pareth

Control logic makes sure that each instruction flows smoothly through the stages of execution, even as different instructions may need different components at the same time. For example, if one instruction is using the ALU while another instruction is being fetched, control logic helps keep everything coordinated without conflicts or delays.

Interaction Between Components:

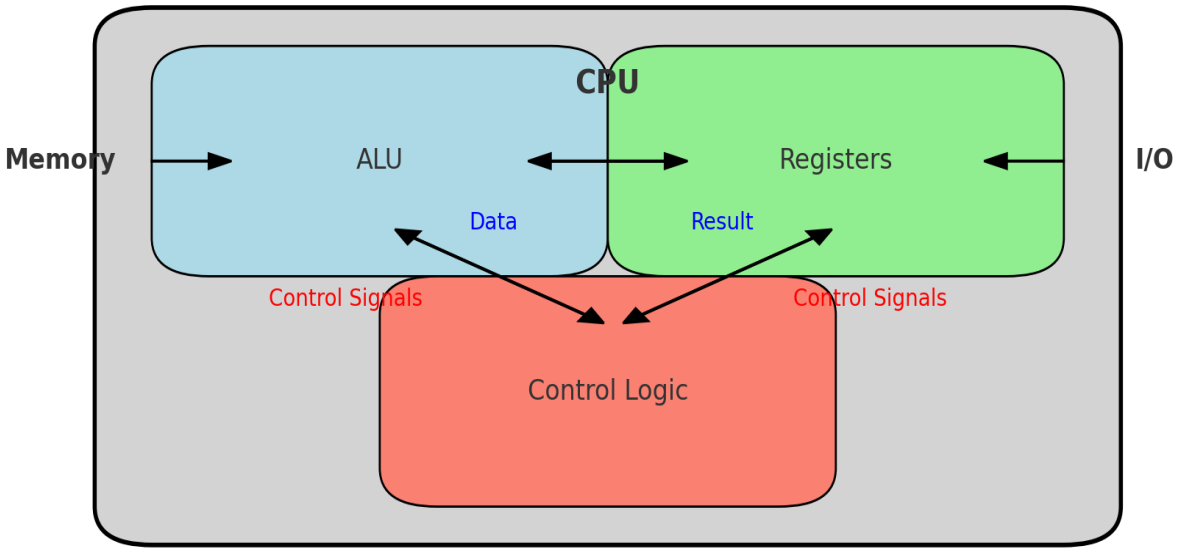
Here's how these components work together during the execution of an instruction:

Fetch: The program counter (PC) register points to the next instruction's address. The control logic fetches this instruction from memory and stores it in the instruction register (IR).

Decode: The control logic decodes the instruction in the IR to determine which operation is needed and which registers or memory locations will be involved.

Execute: The ALU performs the necessary operation (e.g., addition or comparison) using data from the specified registers.

Store: The result from the ALU is stored back in a register or sent to memory if needed.



- The CPU includes **14 general-purpose registers**.
- These registers act as the interface between the CPU and memory during instruction execution.

3. **Memory:**
   - Memory in Y86-64 is **byte-addressable**.
   - Instructions and data are stored in a **unified memory space**.
   - The CPU interacts with memory during the fetch, decode, and execute stages.
4. **Instruction Set Architecture (ISA):**

The Y86-64 ISA includes a simplified set of instructions:

- **Arithmetic operations**:
  - `addq`, `subq`, `andq`, `xorq`.
- **Data movement**:
  - `irmovq`, `rmmovq`, `mrmovq`.
- **Control flow**:
  - `jmp`, `je`, `jne`, `call`, `ret`.
- **Stack operations**:
  - `pushq`, `popq`.

These instructions are simpler than the full x86-64 ISA but cover key concepts such as control flow, data movement, and arithmetic.

1. Registers:

Y86-64 provides 14 general-purpose 64-bit registers, used for various purposes such as data storage, address computation, and stack management:

- **General-purpose registers**:
  - `%rax`: Accumulator register, often used in arithmetic operations.
  - `%rcx`: Counter register, used in loops or shift/rotate instructions.
  - `%rdx`: Data register, used for input/output or as a secondary accumulator.
  - `%rbx`: Base register, used to hold base addresses.
  - `%rsp`: Stack Pointer, tracks the top of the stack.
  - `%rbp`: Base Pointer, used to manage stack frames during function calls.
  - `%rsi`: Source Index, used for string operations and pointer source.
  - `%rdi`: Destination Index, used for string operations and pointer destination.
  - `%r8` to `%r14`: Additional general-purpose registers available in the 64-bit architecture.
- **Special-purpose registers**:
  - `%rsp`: Serves as the stack pointer, critical for stack operations like `pushq` and `popq`.
  - `%rbp`: Acts as the base pointer for managing stack frames.

These registers provide sufficient flexibility to support the Y86-64 instruction set while simplifying the complexity of the full x86-64 architecture.

Instruction Set:

The Y86-64 instruction set is a subset of x86-64 and includes the following basic instructions:

- **Data movement**:
  - `irmovq`, `rmmovq`, `mrmovq` (move between memory and registers).
- **Arithmetic and logic**:
  - `addq`, `subq`, `andq`, `xorq` (basic operations on registers).
- **Control flow**:

Dr.Sureshpareth

  - `jmp`, `je`, `jne`, `jge`, `jl`, `call`, `ret`.
- **Stack operations**:
  - `pushq`, `popq`.

Y86-64 supports basic stack operations through two instructions:

pushq rA: Pushes the value in register rA onto the stack.

This operation decrements the stack pointer `%rsp` (which points to the top of the stack) by 8 (since each stack entry is 64 bits or 8 bytes) and then stores the value from register rA at the new top of the stack.

popq rA: Pops the value from the top of the stack into register rA.

This operation loads the value from the address pointed to by %rsp into register rA and then increments %rsp by 8 to remove the value from the stack, effectively moving the stack pointer up.

Instruction Format:

Y86-64 instructions are encoded in a compact binary format for efficient processing.

- **Instruction fields**:
  1. **Opcode**: Identifies the type of operation (e.g., `addq`, `jmp`).
  2. **Register specifiers**: Specifies source and destination registers.
  3. **Immediate values**: Constants used directly in calculations or memory addresses.
  4. **Memory addresses**: Used in load/store instructions.
- **Fixed-length encoding**:
  - Instructions typically require 1-10 bytes, depending on the complexity.

Y86-64 Registers and Their Opcodes:

Each register in Y86-64 is assigned a unique 4-bit code for encoding purposes. These codes are used in instructions to specify source and destination registers.

Register	Opcode
%rax	0x0
%rcx	0x1
%rdx	0x2
%rbx	0x3
%rsp	0x4
%rbp	0x5
%rsi	0x6
%rdi	0x7
%r8	0x8
%r9	0x9
%r10	0xA
%r11	0xB
%r12	0xC
%r13	0xD
%r14	0xE

Notes:

Dr.Sureshparetti

1. The **opcode for a register** is used in fields specifying *rA* (source register) or *rB* (destination register) in the Y86-64 instruction format.
2. *0xF* is reserved as a special code to indicate **no register** (e.g., in *irmovq* where only one register is used).

Y86-64 Instruction Set and Opcodes:

This section provides a detailed overview of the Y86-64 instructions, their opcodes, and formats.

1. **Data Movement Instructions**
   - `halt`: Halts the processor.
     - *Opcode*: `0x00`
     - *Format*: `halt`
   - `nop`: No operation; does nothing.
     - *Opcode*: `0x10`
     - *Format*: `nop`
   - `rrmovq rA, rB`: Move value from register `rA` to register `rB`.
     - *Opcode*: `0x20`
     - *Format*: `rrmovq rA, rB`
   - `irmovq V, rB`: Move immediate value `V` to register `rB`.
     - *Opcode*: `0x30`
     - *Format*: `irmovq V, rB`
   - `rmmovq rA, D(rB)`: Move value from register `rA` to memory at address `D + rB`.
     - *Opcode*: `0x40`
     - *Format*: `rmmovq rA, D(rB)`
   - `mrmovq D(rB), rA`: Move value from memory at address `D + rB` to register `rA`.
     - *Opcode*: `0x50`
     - *Format*: `mrmovq D(rB), rA`
2. **Arithmetic and Logical Instructions**
   - `addq rA, rB`: Add value in register `rA` to value in register `rB`; store result in `rB`.
     - *Opcode*: `0x60`
     - *Function Code*: `0x0`
     - *Format*: `addq rA, rB`
   - `subq rA, rB`: Subtract value in register `rA` from value in register `rB`; store result in `rB`.
     - *Opcode*: `0x61`
     - *Function Code*: `0x1`
     - *Format*: `subq rA, rB`
   - `andq rA, rB`: Perform bitwise AND between values in registers `rA` and `rB`; store result in `rB`.

Dr.Sureshpareth

  - *Opcode*: `0x62`
  - *Function Code*: `0x2`
  - *Format*: `andq rA, rB`
- `xorq rA, rB`: Perform bitwise XOR between values in registers `rA` and `rB`; store result in `rB`.
  - *Opcode*: `0x63`
  - *Function Code*: `0x3`
  - *Format*: `xorq rA, rB`

3. **Control Flow Instructions**
   - `jmp Dest`: Unconditional jump to address *Dest*.
     - *Opcode*: *0x70*
     - *Function Code*: *0x0*
     - *Format*: *jmp Dest*
   - `jle Dest`: Jump to address *Dest* if less than or equal (signed).
     - *Opcode*: *0x71*
     - *Function Code*: *0x1*
     - *Format*: *jle Dest*
   - `jl Dest`: Jump to address *Dest* if less than (signed).
     - *Opcode*: *0x72*
     - *Function Code*: *0x2*
     - *Format*: *jl Dest*
   - `je Dest`: Jump to address *Dest* if equal.
     - *Opcode*: *0x73*
     - *Function Code*: *0x3*
     - *Format*: *je Dest*
   - `jne Dest`: Jump to address *Dest* if not equal.
     - *Opcode*: *0x74*
     - *Function Code*: *0x4*
     - *Format*: *jne Dest*
   - `jge Dest`: Jump to address *Dest* if greater than or equal (signed).
     - *Opcode*: *0x75*
     - *Function Code*: *0x5*
     - *Format*: *jge Dest*
   - `jg Dest`: Jump to address *Dest* if greater than (signed).
     - *Opcode*: *0x76*
     - *Function Code*: *0x6*

Dr.Sureshpareth

    - *Format*: *jg Dest*
  - `call Dest`: Call subroutine at address *Dest*.
    - *Opcode*: *0x80*
    - *Format*: *call Dest*
  - `ret`: Return from subroutine.
    - *Opcode*: *0x90*
    - *Format*: *ret*

4. **Stack Operations**
   - `pushq rA`: Push value in register *rA* onto the stack.
     - *Opcode*: *0xA0*
     - *Format*: *pushq rA*
   - `popq rA`: Pop value from the stack into register *rA*.
     - *Opcode*: *0xB0*
     - *Format*: *popq rA*
5. **Conditional Move Instructions**
   - `cmovle rA, rB`: Move value from *rA* to *rB* if less than or equal (signed).
     - *Opcode*: *0x21*
     - *Function Code*: *0x1*
     - *Format*: *cmovle rA, rB*
   - `cmovl rA, rB`: Move value from *rA* to *rB* if less than (signed).
     - *Opcode*: *0x22*
     - *Function Code*: *0x2*
     - *Format*: *cmovl rA, rB*
   - `cmove rA, rB`: Move value from *rA* to *rB* if equal.
     - *Opcode*: *0x23*
     - *Function Code*: *0x3*
     - *Format*: *cmove rA, rB*
   - `cmovne rA, rB`: Move value from *rA* to *rB* if not equal.
     - *Opcode*: *0x24*
     - *Function Code*: *0x4*
     - *Format*: *cmovne rA, rB*
   - `cmovge rA, rB`: Move value from *rA* to *rB* if greater than or equal (signed).
     - *Opcode*: *0x25*

The following table summarizes the Y86-64 instructions, their opcodes, function codes, and formats:

Dr.Sureshpareth

Instruction	Opcode | Function Code | Format		
halt	0x00	N/A	halt
nop	0x10	N/A	nop
rrmovq rA, rB	0x20	0x0	rrmovq rA, rB
irmovq V, rB	0x30	N/A	irmovq V, rB
rmmovq rA, D(rB)	0x40	N/A	rmmovq rA, D(rB)
mrmovq D(rB), rA	0x50	N/A	mrmovq D(rB), rA
addq rA, rB	0x60	0x0	addq rA, rB
subq rA, rB	0x60	0x1	subq rA, rB
andq rA, rB	0x60	0x2	andq rA, rB
xorq rA, rB	0x60	0x3	xorq rA, rB
jmp Dest	0x70	0x0	jmp Dest
jle Dest	0x70	0x1	jle Dest
jl Dest	0x70	0x2	jl Dest
je Dest	0x70	0x3	je Dest
jne Dest	0x70	0x4	jne Dest
jge Dest	0x70	0x5	jge Dest
jg Dest	0x70	0x6	jg Dest
call Dest	0x80	N/A	call Dest
ret	0x90	N/A	ret
pushq rA	0xA0	N/A	pushq rA
popq rA	0xB0	N/A	popq rA
cmovle rA, rB	0x20	0x1	cmovle rA, rB
cmovl rA, rB	0x20	0x2	cmovl rA, rB
cmove rA, rB	0x20	0x3	cmove rA, rB
cmovne rA, rB	0x20	0x4	cmovne rA, rB
cmovge rA, rB	0x20	0x5	cmovge rA, rB
cmovg rA, rB	0x20	0x6	cmovg rA, rB

In Y86-64, the instruction format size (or encoding) is also simplified, with each instruction having a fixed number of bytes based on its type. The format is designed to be compact and straightforward, making it easier to work with and simulate in educational environments. Here's a breakdown of the typical sizes for Y86-64 instructions.

Y86-64 Instruction Format and Size

In Y86-64, the instruction encoding generally falls into three main formats based on the type of instruction:

Single-Byte Instructions:

Size: 1 byte

Examples: halt, nop

These instructions perform simple operations and don't require operands.

Two-Byte Instructions:

Size: 2 bytes

Examples: rrmovq %rA, %rB, pushq %rA, popq %rA, addq %rA, %rB, subq %rA, %rB, andq %rA, %rB, xorq %rA, %rB

The first byte specifies the opcode, and the second byte specifies the registers involved, typically with one nibble (4 bits) per register.

Dr.Sure

Ten-Byte Instructions:

Size: 10 bytes

Examples: irmovq $V, %rB, rmmovq %rA, D(%rB), mrmovq D(%rB), %rA, jmp Dest, jle Dest, jl Dest, je Dest, jne Dest, jge Dest, jg Dest, call Dest

These instructions involve immediate values, memory addresses, or displacements, so they are longer.

Format:

1 byte for the opcode

1 byte for register encoding (if registers are involved)

8 bytes for the immediate value, destination, or displacement ($V, Dest, D).

Instruction Sizes:

Table 1: Summary of Instruction Sizes

Instruction Type	Examples	Size
Single-byte	halt, nop	1 byte
Two-byte	rrmovq, pushq, popq, addq, etc.	2 bytes
Ten-byte	irmovq, rmmovq, mrmovq, jmp	10 bytes

Example Encodings

Single-Byte Instruction:

halt (opcode 0x00): Only 1 byte.

Two-Byte Instruction:

addq %rax, %rbx (opcode 0x60): 2 bytes, with one byte for the opcode and one byte to encode the source and destination registers.

Ten-Byte Instruction:

irmovq $5, %rax (opcode 0x30): 10 bytes, including 1 byte for the opcode, 1 byte for register encoding, and 8 bytes for the immediate value.

This structure keeps the Y86-64 instruction set simple and predictable, making it ideal for learning about CPU instruction execution without the added complexity of variable-length instructions, as found in x86-64.

Assembly Code and Machine Code:

- **Assembly Code**:
  - A human-readable representation of instructions (e.g., `addq %rax, %rbx`).
  - Includes labels, symbols, and pseudo-operations to simplify programming.
- **Machine Code**:
  - Binary representation of instructions for execution by the processor.
  - Assembly code is translated into machine code using an assembler.

The CPU (Central Processing Unit) is the core component responsible for executing instructions in a computer program. When a program runs, the CPU follows a series of steps to fetch, decode, and execute each instruction in the code. Here's a detailed look at the CPU's role in code execution.

Dr.Suresh Kumar Retha

1. Fetching Instructions

The CPU begins by fetching the next instruction to execute from memory. The address of this instruction is held in a special register called the Program Counter (PC).

Program Counter (PC): The PC keeps track of the current instruction's location in memory and increments after each instruction is fetched to move to the next one.

Fetch Cycle: This step involves accessing the system's memory to retrieve the binary data that represents the instruction.

2. Decoding Instructions

Once the CPU fetches an instruction, it needs to interpret or decode it. The decoding process determines what type of instruction it is and what actions the CPU needs to take.

Instruction Decoder: The CPU has a component called the instruction decoder, which reads the binary data and translates it into signals that other parts of the CPU can understand.

The decoded instruction may specify:

Operation: What action to perform, such as addition, data movement, or conditional branching.

Operands: The data required for the operation, which could be stored in registers, memory, or provided as an immediate value.

Destination: Where to store the result of the operation, often a register or a specific memory location.

3. Executing Instructions

After decoding, the CPU proceeds to execute the instruction. This step involves carrying out the actual operation specified by the instruction, which can involve arithmetic calculations, data movement, or logical operations.

Key Components in Execution:

Arithmetic Logic Unit (ALU): The ALU performs arithmetic (like addition or subtraction) and logical operations (like AND, OR) as required by the instruction.

Registers: Small, fast storage locations within the CPU hold data that the ALU can access immediately for computations.

Control Unit: The control unit coordinates between the ALU, registers, and other components to ensure that the instruction is executed correctly.

Data Movement: Some instructions involve moving data between memory and registers or transferring data between registers within the CPU.

4. Writing Back Results

After executing an instruction, the CPU often needs to store the result. This step is sometimes called the write-back stage.

The result can be stored in a register (for immediate access in subsequent instructions) or written back to memory.

The write-back ensures that the result of each instruction is saved in a predictable location, ready for the next instruction or function call.

5. Managing Control Flow and Branching

Not all instructions proceed sequentially. Some instructions, like function calls, conditional statements, and loops, require the CPU to change the flow of execution.

Dr.Sureshparath

Conditional Branching: Based on a condition (e.g., if statements in code), the CPU may jump to a different memory address to execute a specific part of the code.

Loops and Jumps: For loops or unconditional jumps, the CPU modifies the program counter (PC) to point to a new instruction location, effectively creating a loop or jumping to a specific line of code.

Function Calls: When a function is called, the CPU saves the return address (the address of the next instruction after the function call) to the stack. After the function completes, the CPU pops this return address from the stack to resume execution.

6. Handling Input/Output (I/O) Operations

Some instructions involve interacting with external devices (e.g., displaying text on a screen or reading data from a keyboard).

System Calls: For these operations, the CPU often executes a system call, which is a request to the operating system to perform I/O tasks.

Interrupts: When an I/O operation completes or an event occurs (like receiving data from the network), an interrupt may signal the CPU to handle this event immediately, temporarily pausing the current code execution.

7. Using the Stack for Function Calls and Local Variables

The stack is a dedicated area in memory used by the CPU to store information temporarily during function calls.

Push and Pop Operations: When a function is called, arguments, local variables, and return addresses are pushed onto the stack. After the function executes, these values are popped off to clean up.

Base Pointer (BP) and Stack Pointer (SP): The CPU uses these pointers to manage the stack. The stack pointer keeps track of the top of the stack, and the base pointer helps manage local variables within each function's frame.

8. The Fetch-Decode-Execute Cycle

The entire process (fetching, decoding, executing, and writing back) forms a continuous loop called the fetch-decode-execute cycle.

This cycle enables the CPU to process each instruction in turn, executing millions or even billions of instructions per second.

Dr.Sureshpareth

5 Example: Running a Simple Program

To see how the CPU processes a simple program, consider the following C program:

```
int main() {
    int a = 5;
    int b = 3;
    int c = a + b;
    return c;
}
```

The steps below illustrate how the CPU fetches, decodes, and executes each instruction:

1. Fetch the `int a = 5;` Instruction:
   - The CPU fetches and decodes this instruction.
   - It determines that the integer *5* needs to be loaded into memory or a register.
2. Execute the *int b = 3;* Instruction:
   - The CPU fetches this instruction.
   - It loads the integer *3* into memory or a register, similar to the previous step.
3. Execute `int c = a + b;` :
   - The CPU fetches the values of *a* and *b* from registers or memory.
   - It sends these values to the Arithmetic Logic Unit (ALU), which performs the addition operation.
   - The resulting value is stored in *c*.
4. **Return the Result**:
   - The program ends by returning the value of *c*.
   - The CPU uses this value to exit cleanly.

This example demonstrates how a CPU processes a simple program step by step, handling data loading, arithmetic operations, and returning results.

Example-1:

Running a Simple Program: CPU Role, Assembly, and Machine Code:

Dr.Suresh

The following explains how the CPU processes a simple C program, the corresponding Y86-64 assembly code, and the resulting machine code.

The example C program:

```c
int main() {
    int a = 5;
    int b = 3;
    int c = a + b;
    return c;
}
```

The CPU processes this program in the following stages:

1. **Translation**: The C program is compiled into Y86-64 assembly code.
2. **Execution**: The assembly code is converted into machine code for execution.
3. **Runtime**:
   - The CPU fetches instructions from memory.
   - It decodes and executes each instruction.
   - It interacts with registers and memory for data storage and retrieval.
   - Arithmetic operations are performed in the Arithmetic Logic Unit (ALU).

The Y86-64 assembly code corresponding to the C program:

```
irmovq $5, %rax         # Load the constant 5 into register %rax (a = 5)
irmovq $3, %rbx         # Load the constant 3 into register %rbx (b = 3)
addq %rax, %rbx         # Add the values in %rax and %rbx, storing the result in %rbx␣
↪(c = a + b)
rrmovq %rbx, %rax       # Move the result (c) from %rbx to %rax for the return value
ret                     # Return from the function
```

The corresponding Y86-64 Machine Code (in hexadecimal):

Instruction	Opcode | Operands | Machine Code	
irmovq $5, %rax	0x30	Register = %rax, Value = 5 | 30F005000000000000
irmovq $3, %rbx	0x30	Register = %rbx, Value = 3 | 30F103000000000000
addq %rax, %rbx	0x60	Registers = %rax, %rbx | 6003
rrmovq %rbx, %rax	0x20	Registers = %rbx, %rax | 2030
ret	0x90	N/A | 90

Step-by-Step Execution

Here is how the CPU processes this program:

1. Instruction 1: `irmovq $5, %rax`
   - **Fetch**: The CPU fetches the opcode *0x30* and decodes it as *irmovq*.
   - **Decode**: Determines it needs to load the immediate value *5* into *%rax*.
   - **Execute**: Stores *5* in the *%rax* register.
2. Instruction 2: `irmovq $3, %rbx`
   - **Fetch**: The CPU fetches *0x30* and decodes it as *irmovq*.

- **Decode**: Determines it needs to load the immediate value *3* into *%rbx*.
- **Execute**: Stores *3* in the *%rbx* register.

3. Instruction 3: `addq %rax, %rbx`
   - **Fetch**: The CPU fetches *0x60* and decodes it as *addq*.
   - **Decode**: Identifies *%rax* as the source and *%rbx* as the destination.
   - **Execute**: Adds the values in *%rax* and *%rbx*, storing the result (*8*) in *%rbx*.
4. Instruction 4: `rrmovq %rbx, %rax`
   - **Fetch**: The CPU fetches *0x20* and decodes it as *rrmovq*.
   - **Decode**: Identifies *%rbx* as the source and *%rax* as the destination.
   - **Execute**: Moves the value *8* from *%rbx* to *%rax*.
5. Instruction 5: `ret`
   - **Fetch**: The CPU fetches *0x90* and decodes it as *ret*.
   - **Execute**: Ends the program and returns the value in *%rax*.

- **Registers Used**: *%rax*, *%rbx*.
- **ALU Operations**: Addition in *addq*.
- **Output**: The program returns *8*, stored in *%rax*, as the result of the addition (*5 + 3*).

This example demonstrates the complete CPU execution for the given C program, from translation to runtime execution.

Example-2:

Multiplication of Two Numbers in Y86-64:

The C program for multiplying two integers:

```c
int main() {
    int a = 4;
    int b = 6;
    int c = a * b;
    return c;
```

Since Y86-64 does not have a direct *mul* instruction, multiplication is implemented using repeated addition in a loop.

The assembly code for multiplying two numbers (*a* and *b*):

```
# Initialize values
irmovq $4, %rax        # Load 4 into %rax (a = 4)
irmovq $6, %rbx        # Load 6 into %rbx (b = 6)
irmovq $0, %rcx        # Initialize %rcx to 0 (c = 0)

# Multiply using repeated addition
andq %rbx, %rbx        # Check if %rbx (b) is zero
je done                # If b == 0, jump to done
loop:
addq %rax, %rcx        # Add %rax (a) to %rcx (c += a)
subq $1, %rbx          # Decrement %rbx (b -= 1)
jne loop               # If b != 0, repeat loop
```

(continues on next page)

(continued from previous page)

```
done:
rrmovq %rcx, %rax       # Move result (c) into %rax for return
ret                     # Return from the function
```

The corresponding machine code for the assembly instructions is shown below:

Instruction	Opcode | Operands | Machine Code		
irmovq $4, %rax	0x30	Immediate = 4, %rax	30F004000000000000
irmovq $6, %rbx	0x30	Immediate = 6, %rbx	30F306000000000000
irmovq $0, %rcx	0x30	Immediate = 0, %rcx	30F100000000000000
andq %rbx, %rbx	0x62	%rbx, %rbx	6233
je done	0x71	Offset = 17	7117
addq %rax, %rcx	0x60	%rax, %rcx	6001
subq $1, %rbx	0x61	%rbx, Immediate = 1	61F301000000000000
jne loop	0x72	Offset = 12	7212
rrmovq %rcx, %rax	0x20	%rcx, %rax	2010
ret	0x90	N/A	90

Encoded Machine Code Sequence:

The machine code sequence for this program is:

1. **Initialize Values**:
   - The CPU loads *4* into *%rax*, *6* into *%rbx*, and *0* into *%rcx*.
2. **Check Loop Condition**:
   - The CPU checks if *%rbx* is zero using *andq %rbx, %rbx*.
   - If *%rbx* is zero, it jumps to *done*.
3. **Execute Loop**:
   - In each iteration:
     - *%rax* (a) is added to *%rcx* (c) using *addq %rax, %rcx*.
     - *%rbx* is decremented by *1* using *subq $1, %rbx*.
   - The loop continues until *%rbx* becomes zero.
4. **Complete Execution**:
   - Once *%rbx* is zero, the CPU moves the result (*c*) in *%rcx* to *%rax* using *rrmovq %rcx, %rax*.
   - The function ends with *ret*.

The program multiplies *4* and *6* using repeated addition. The result, *24*, is stored in *%rax* and returned as the output.

Finding the Greatest of Two Numbers in Y86-64:

The C program for finding the greatest of two numbers:

```
int main() {
    int a = 10;
    int b = 20;
```

(continues on next page)

Dr.Sureshpa

(continued from previous page)

```
    int max;

    if (a > b) {
        max = a;
    } else {
        max = b;
    }

    return max;
}
```

The corresponding assembly code for the C program:

```
# Initialize values
irmovq $10, %rax        # Load 10 into %rax (a = 10)
irmovq $20, %rbx        # Load 20 into %rbx (b = 20)
irmovq $0, %rcx         # Initialize %rcx to 0 (max = 0)

# Compare a and b

    rrmovq %rax %rdx
subq %rbx, %rax         # Compute a - b (sets condition codes)
jg greater              # If a > b, jump to label "greater"

# b is greater or equal
rrmovq %rbx, %rcx       # Move b into %rcx (max = b)
jmp done                # Skip the next block

greater:
rrmovq %rdx, %rcx       # Move a into %rcx (max = a)

done:
rrmovq %rcx, %rax       # Move result (max) into %rax for return
ret                     # Return from the function
```

The corresponding machine code for the assembly instructions:

Instruction	Opcode | Operands | Machine Code		
irmovq $10, %rax	0x30	Immediate = 10, %rax	30F00A000000000000
irmovq $20, %rbx	0x30	Immediate = 20, %rbx	30F314000000000000
irmovq $0, %rcx	0x30	Immediate = 0, %rcx	30F100000000000000
rrmovq %rax, %rdx	0x20	%rax, %rdx	2002
subq %rbx, %rax	0x61	%rbx, %rax	6130
jg greater	0x76	Offset = 09	7609
rrmovq %rbx, %rcx	0x20	%rbx, %rcx	2031
jmp done	0x70	Offset = 05	7005
rrmovq %rdx, %rcx	0x20	%rdx, %rcx	2021
rrmovq %rcx, %rax	0x20	%rcx, %rax	2010
ret	0x90	N/A	90

- **Input**: $a = 10, b = 20$.
- **Output**: The greater value is 20, returned in *%rax*.

6

Role of the Header in Executable and Execution of Subtraction in Y86-64

Dr.Sureshpare

This document explains the role of the header in an executable file created by the compiler, the OS's role, and the execution of a C program for subtraction in the Y86-64 architecture.

Role of the Compiler in producing the Executable:

Compiler Steps:

Compile:

The compiler generates Y86-64 assembly from the C code.

Assemble:

The assembler translates the assembly code into machine code.

Link:

The linker adds a header to the binary file and organizes sections (e.g., code, data, stack).

When a compiler creates an executable, it includes a **header** section. The header provides metadata necessary for the OS to load and execute the program correctly.

Key components of the header include:

1. **Magic Number**:
   - Identifies the file type and ensures the binary is valid.
   - Example: *0x79683634* ("y86-64").
2. **Entry Point**:
   - Specifies the memory address of the program's starting instruction.
   - Example: *0x0000000000000010*.
3. **Section Information**:
   - **Code Offset**: Start address of the code section.
   - **Data Offset**: Start address of the data section.
   - **Stack Size**: Size allocated for the stack.
4. **Program Metadata**:

- Provides details such as program size and target architecture.

Example Header:

An example header for a Y86-64 executable might look like this:

Field	Example Value
Magic Number	*0x79683634* ("y86-64")
Entry Point	*0x0000000000000010*
Code Offset	*0x00000010*
Data Offset	*0x00000050*
Stack Size	*0x00001000*
Program Size	*0x00000200*

6.1 C Code for Subtraction

The C program for subtracting two numbers:

```c
int main() {
    int a = 15;
    int b = 7;
    int result = a - b;
    return result;
}
```

The assembly code with a header for the C program:

```
# HEADER SECTION
.header:
    .magic 0x79683634       # Magic number ("y86-64")
    .entry 0x10             # Entry point address
    .code_offset 0x10       # Code section begins at 0x10
    .data_offset 0x50       # Data section begins at 0x50
    .stack_size 0x1000      # Stack size allocated
    .program_size 0x200     # Total program size

# CODE SECTION
.text:
    irmovq $15, %rax        # Load 15 into %rax (a = 15)
    irmovq $7, %rbx         # Load 7 into %rbx (b = 7)
    subq %rbx, %rax         # Compute a - b, store result in %rax (result = a - b)
    ret                     # Return result in %rax
```

The corresponding machine code for the program, including the header:

Dr. Gareth

Field/Instruction	Machine Code
Magic Number	*0x79683634*
Entry Point	*0x0000000000000010*
Code Offset	*0x00000010*
Data Offset	*0x00000050*
Stack Size	*0x00001000*
Program Size	*0x00000200*
irmovq $15, %rax	*30F00F000000000000*
irmovq $7, %rbx	*30F307000000000000*
subq %rbx, %rax	*6130*
ret	*90*

Execution Steps

Here's how the OS and CPU execute the program:

1. **Compilation**:
   - The compiler generates Y86-64 assembly code and converts it to machine code.
   - The linker adds a header to the binary and organizes sections.
2. **Program Loading**:
   - The OS reads the header to:
     - Verify the executable format.
     - Determine the program's entry point.
     - Load the code and data sections into memory.
3. **Execution**:
   - The OS initializes the Program Counter (PC) to the entry point (*0x10*).
   - The CPU executes instructions step by step:
     1. `irmovq $15, %rax`: Loads *15* into *%rax*.
     2. `irmovq $7, %rbx`: Loads *7* into *%rbx*.
     3. `subq %rbx, %rax`: Subtracts *7* from *15*, storing *8* in *%rax*.
     4. `ret`: Returns the result (*8*) to the OS.
4. **Cleanup**:
   - The OS retrieves the result from *%rax* and terminates the process.

The complete machine code sequence for this program:

```
30F00F000000000000  # irmovq $15, %rax
30F307000000000000  # irmovq $7, %rbx
6130                # subq %rbx, %rax
90                  # ret
```

Output

- **Input**: `a = 15`, `b = 7`
- **Output**: *result = 8*, stored in *%rax*

Dr.Sureshpareth

The Header's Role:

1. **Program Loading**:
   - The OS reads the header to load the binary and initialize the CPU.
2. **Section Management**:
   - The header provides offsets for code, data, and stack sections.
3. **Execution Control**:
   - The header specifies the entry point for the program.
4. **Resource Management**:
   - Ensures proper execution and cleanup after the program ends.

The header ensures seamless interaction between the OS and the program, enabling correct execution of instructions.

Dr.Sureshpareth

7

Role of the OS and Execution of Subtraction in Y86-64

This document explains the role of the operating system (OS) in executing a program on the Y86-64 architecture. It also details the execution process for a C program that subtracts two numbers.

Role of the Operating System in Y86-64

The OS plays a critical role in managing the execution of a program on Y86-64:

1. **Program Loading**:
   - Reads the program's binary file and loads it into memory.
   - Allocates memory for code, data, and stack sections.
2. **Process Management**:
   - Initializes the Program Counter (PC) to point to the entry point of the program.
   - Manages context switching if multiple programs are running.
3. **System Calls**:
   - Provides access to essential system resources such as I/O and memory management.
   - Handles *exit* or *return* calls to clean up resources after execution.
4. **Interrupt Handling**:
   - Handles interrupts generated by the program or external devices.
5. **Simulation/Emulation**:
   - For Y86-64, the OS is typically simulated, managing execution within an emulator.

The C program for subtracting two numbers:

```c
int main() {
    int a = 15;
    int b = 7;
    int result = a - b;
    return result;
}
```

The corresponding Y86-64 assembly code:

Dr.Sureshpare

```
# Initialize values
irmovq $15, %rax        # Load 15 into %rax (a = 15)
irmovq $7, %rbx         # Load 7 into %rbx (b = 7)

# Perform subtraction
subq %rbx, %rax         # Compute a - b, store result in %rax (result = a - b)

# Return result
ret                     # Return from the function with result in %rax
```

The OS and CPU execute the program as follows:

1. **Compilation**:
   - The C compiler translates the code into Y86-64 assembly code and converts it into machine code.
   - Example machine code:

```
30F00F000000000000  # irmovq $15, %rax
30F307000000000000  # irmovq $7, %rbx
6130                # subq %rbx, %rax
90                  # ret
```

2. **Program Loading**:
   - The OS loads the machine code into memory.
   - Sets up:
     - **Code Section**: Contains machine code instructions.
     - **Data Section**: Allocates memory for variables.
     - **Stack Section**: Prepares a stack for function calls and returns.
3. **Process Initialization**:
   - The OS initializes the CPU:
     - Loads the Program Counter (PC) with the program's starting address (entry point).
     - Initializes general-purpose registers (*%rax*, *%rbx*, etc.).
     - Sets up the stack pointer (*%rsp*).
4. **Instruction Execution**:
   - The CPU executes instructions step by step:
     - `irmovq $15, %rax`: Loads the value *15* into *%rax*.
     - `irmovq $7, %rbx`: Loads the value *7* into *%rbx*.
     - `subq %rbx, %rax`: Subtracts the value in *%rbx* (*7*) from *%rax* (*15*) and stores the result (*8*) in *%rax*.
     - `ret`: Returns from the function.
5. **System Call to Return**:
   - The *ret* instruction signals the OS to handle the program's exit.
   - The OS retrieves the result from *%rax* and terminates the process.

Dr.Sureshpretin

6. **Cleanup**:
   - The OS deallocates memory and resets the CPU state for the next process.

The machine code for the assembly instructions is as follows:

Instruction	Opcode | Operands | Machine Code		
irmovq $15, %rax	0x30	Immediate = 15, %rax	30F00F000000000000
irmovq $7, %rbx	0x30	Immediate = 7, %rbx	30F307000000000000
subq %rbx, %rax	0x61	%rbx, %rax	6130
ret	0x90	N/A	90

Encoded Machine Code Sequence

The complete machine code sequence:

```
30F00F000000000000  # irmovq $15, %rax
30F107000000000000  # irmovq $7, %rbx
6123                # subq %rbx, %rax
90                  # ret
```

Output of the Program

- **Input**: *a = 15, b = 7*
- **Output**: *result = 8*, stored in *%rax*

The OS's Role:

1. **Program Loading**: Loads the binary into memory and initializes CPU registers.
2. **Process Execution**: Manages execution of instructions and CPU state.
3. **System Call Handling**: Processes the *ret* instruction and terminates the program.
4. **Resource Management**: Allocates and deallocates memory and CPU time for the program.

The OS ensures seamless execution of the program while managing all necessary resources.

Dr.Sureshpatil

8 Instruction Execution in SEQ

Each instruction in SEQ is executed in five stages:

1. Fetch (F):

Read the instruction from memory at the address in the PC.

Increment the PC to point to the next instruction.

2. Decode (D):

Decode the instruction.

Read source operands from the register file.

3. Execute (E):

Perform the operation specified by the instruction (e.g., addition, subtraction).

Compute memory addresses for load/store instructions.

4. Memory (M):

Access data memory for load (mrmovq) or store (rmmovq) instructions.

Bypass this stage for arithmetic instructions.

5. Write-Back (W):

Write results back to the destination register (if applicable).

Example: SEQ Instruction Execution:

Consider the following Y86-64 program:

```
irmovq $5, %rax         # Load immediate 5 into %rax
irmovq $3, %rbx         # Load immediate 3 into %rbx
addq %rbx, %rax         # Add %rbx to %rax
halt                    # Stop execution
```

Execution in SEQ:

Dr.Sureshpare

Table 1: Execution in SEQ

Cycle	Instruction	Fetch	Decode	Execute	Memory	Write-Back
1	irmovq $5, %rax	Fetch opcode	Decode immediate	Load immediate	N/A	Write to %rax
2	irmovq $3, %rbx	Fetch opcode	Decode immediate	Load immediate	N/A	Write to %rbx
3	addq %rbx, %rax	Fetch opcode	Decode operands	Add %rbx to %rax	N/A	Write to %rax
4	halt	Fetch opcode	Decode halt	Stop execution	N/A	N/A

Control Logic in SEQ

The control logic in SEQ generates signals to direct the flow of data through the processor. Key control signals include:

Instruction Type:

Determines whether the instruction is an arithmetic operation, memory operation, or control flow instruction.

ALU Control:

Specifies the operation the ALU should perform (e.g., addition, subtraction).

Memory Control:

Indicates whether to read or write to memory.

Write-Back Enable:

Specifies whether to write the result to the register file.

Dr.Sureshpare

9 Pipeline

Pipeline architecture in Y86-64 is a simplified model designed to illustrate the concept of instruction pipelining without the complexity of modern, highly optimized CPUs. Pipelining in Y86-64 breaks down instruction execution into stages, allowing multiple instructions to be processed simultaneously at different stages in the pipeline. This approach increases instruction throughput by overlapping the execution of multiple instructions.

Basics of Pipelining:

In general, pipelining allows a CPU to start processing a new instruction before the previous one has completed. By splitting instruction execution into stages, each stage can work on a different instruction simultaneously, maximizing the use of CPU resources and increasing the overall instruction execution rate.

Y86-64 uses a **5-stage pipeline** similar to modern CPUs:

Fetch (F): The instruction is fetched from memory at the address specified by the Program Counter (PC).

Decode (D): The fetched instruction is decoded to identify the operation, source, and destination operands.

Execute (E): The ALU (Arithmetic Logic Unit) performs the operation specified by the instruction (e.g., addition, subtraction).

Memory (M): The instruction may read from or write to memory if it involves a memory access.

Writeback (W): The result of the operation is written back to the destination register, completing the instruction.

Each instruction goes through these five stages in sequence, but because the pipeline is staggered, multiple instructions can be in different stages at the same time.

Without pipelining, each instruction would complete all five stages before the next one starts, which would take 15 cycles in total (5 stages x 3 instructions).

With pipelining, the CPU can overlap these instructions:

Dr.Sureshpare

Table 1: Pipelining Stages

Cycle	Instruction 1 (irmovq $10, %rax)	Instruction 2 (irmovq $5, %rbx)	Instruction 3 (addq %rbx, %rax)
1	Fetch		
2	Decode	Fetch	
3	Execute	Decode	Fetch
4	Memory	Execute	Decode
5	Writeback	Memory	Execute
6		Writeback	Memory
7			Writeback

Handling Pipeline Hazards:

While pipelining increases performance, it can introduce challenges, or pipeline hazards, that need to be managed:

Data Hazards: When instructions depend on the results of previous instructions.

For example, `addq %rbx, %rax` depends on the result of `irmovq $5, %rbx`. The CPU may use forwarding or stalling techniques to handle this.

Control Hazards: When the pipeline has to handle a branch instruction (e.g., jmp), and it's not clear which instruction should be fetched next.

Simple methods to handle control hazards include stalls or branch prediction (though Y86-64 often uses stalling for simplicity).

Structural Hazards: When hardware resources are not available for all instructions at the same time (rare in Y86-64 due to its simplicity).

In a pipelined architecture like Y86-64, a data hazard occurs when an instruction depends on the result of a previous instruction that has not yet completed its execution. Forwarding and stalling are two techniques the CPU can use to manage these data dependencies and ensure that instructions execute correctly.

Understanding Data Hazards:

A data hazard happens when an instruction needs data that hasn't been fully processed and written back by a preceding instruction in the pipeline. This usually occurs because of the overlapping nature of pipelining, where different stages of multiple instructions are in progress simultaneously.

Example of a Data Hazard:

Consider these instructions in the Y86-64 pipeline:

irmovq $10, %rax – Loads the immediate value 10 into %rax.

addq %rax, %rbx – Adds %rax to %rbx.

In a pipelined CPU, the addq %rax, %rbx instruction might start executing before the irmovq $10, %rax has completed. This means that %rax may not contain the correct value when addq tries to use it, leading to a data hazard.

Techniques to Handle Data Hazards:

To handle data hazards, CPUs can use forwarding or stalling:

1. Forwarding (Data Forwarding or Bypassing):

Dr.Suresh parey

Forwarding is a technique where the CPU directly forwards the output of one instruction to a subsequent instruction before it is officially written back to the register.

How It Works: Instead of waiting for the irmovq $10, %rax instruction to complete all stages, the CPU takes the result from the Execute (E) stage (or another appropriate stage) and sends it directly to the Execute stage of the dependent instruction (addq %rax, %rbx), bypassing the normal pipeline flow.

Example:

After irmovq $10, %rax completes its Execute stage, the value 10 is available.

The CPU forwards this value directly to the Execute stage of the addq %rax, %rbx instruction.

This allows addq %rax, %rbx to proceed without waiting for irmovq $10, %rax to fully complete its Writeback stage.

Benefits of Forwarding:

Reduces Stalls: Forwarding avoids unnecessary pipeline stalls by making data available as soon as it is computed.

Increases Efficiency: Instructions can proceed without waiting for data to be written back to registers, improving pipeline performance.

2. Stalling (Pipeline Stalling):

Stalling (or inserting bubbles) temporarily halts the progress of instructions in the pipeline to wait for the required data to be ready. This involves adding NOP (no operation) instructions, effectively pausing the pipeline for a cycle.

How It Works: When a data hazard is detected, the CPU introduces a stall by inserting one or more NOP instructions (bubbles) in the pipeline.

This allows the pipeline to delay the dependent instruction until the previous instruction completes and writes back its result.

Example:

In the previous example, if addq %rax, %rbx needs the value from irmovq $10, %rax, the CPU could insert a stall.

This would delay the addq %rax, %rbx instruction until %rax is fully updated.

Drawbacks of Stalling::

Reduces Efficiency: Stalling slows down the pipeline because instructions have to wait for earlier instructions to complete, which reduces overall throughput.

Should Be Minimized: While necessary in some cases, stalling is generally less efficient than forwarding and is used only when forwarding is not possible.

Y86-64 Pipeline Control Logic:

The control logic in a pipelined Y86-64 CPU manages the flow of instructions through each stage. It handles hazards and ensures that data dependencies are respected. In a classroom or educational simulator, control logic may be simplified to show basic stalling or forwarding to address hazards.

Y86-64 Pipelining:

Pipeline Stages: Fetch, Decode, Execute, Memory, Writeback.

Parallel Execution: Instructions move through the pipeline simultaneously, each in a different stage.

Dr.Sureshpareth

Hazard Management: Uses stalling and forwarding to resolve data dependencies and control hazards.

Simplified Model: Y86-64's pipeline is a simplified, educational example designed to demonstrate the basics of pipelining without the complexity of real-world CPUs.

The pipelined Y86-64 model illustrates fundamental pipelining concepts, helping to build an understanding of how more advanced CPU architectures manage multiple instructions efficiently.

- **Pipelining benefits**:
  - Increases instruction throughput by overlapping the execution of multiple instructions.

Simplifications in Y86-64:

Y86-64 is intentionally simplified to focus on core concepts:

- No complex addressing modes.
- No advanced features like SIMD or floating-point operations.
- Fewer instructions and simplified pipeline design make it ideal for educational purposes.

The Y86-64 architecture provides a foundational understanding of modern processor design by simplifying the x86-64 model. Key components include:

- A well-defined set of registers.
- A straightforward instruction set.
- Compact instruction encoding.
- Assembly-to-machine code translation.
- Pipeline concepts with solutions to hazards.

These elements demonstrate the principles of computer systems in an approachable and comprehensible manner.

Dr.Sureshpareth

10

Pipeline Hazards and Forwarding in Y86-64

The C program for summing three variables:

```
int main() {
    int a = 5;
    int b = 7;
    int c = a + b;
    int d = c + 2;
    return d;
```

In this example, the computation of d depends on the result of c, introducing a **data hazard**.

The corresponding Y86-64 assembly code:

```
irmovq $5, %rax        # Load 5 into %rax (a = 5)

irmovq $7, %rbx        # Load 7 into %rbx (b = 7)

addq %rbx, %rax        # Add %rbx (b) to %rax (a), result in %rax (c)

irmovq $2, %rcx        # Load 2 into %rcx

addq %rcx, %rax        # Add %rcx (2) to %rax (c), result in %rax (d)

ret                    # Return %rax (d)
```

10.1 Pipeline Stages Without Forwarding

Without forwarding, the pipeline introduces **stalls** due to data dependency between instructions.

Cycle	Stage for *irmovq $5, %rax*	Stage for *irmovq $7, %rbx*	Stage for *addq %rbx, %rax*	Stage for *addq %rcx, %rax*
1	Fetch (F)			
2	Decode (D)	Fetch (F)		
3	Execute (E)	Decode (D)	Fetch (F)	
4	Memory (M)	Execute (E)	Decode (D)	Fetch (F)
5	Write-back (W)	Memory (M)	Execute (E)	Decode (D)
6		Write-back (W)	Memory (M)	**STALL**
7			Write-back (W)	Decode (D)
8				Execute (E)

Explanation of Stalls:

- Instruction 5 (*addq %rcx, %rax*) depends on the result of Instruction 3 (*addq %rbx, %rax*).
- Instruction 5 must wait until Instruction 3 completes its Write-back (W) stage, causing a stall in Cycle 6.

Pipeline Stages with Forwarding

With **data forwarding**, the stall is avoided by passing the result from the Execute (E) stage of Instruction 3 directly to the Execute (E) stage of Instruction 5.

Cycle	Stage for *irmovq $5, %rax*	Stage for *irmovq $7, %rbx*	Stage for *addq %rbx, %rax*	Stage for *addq %rcx, %rax*
1	Fetch (F)			
2	Decode (D)	Fetch (F)		
3	Execute (E)	Decode (D)	Fetch (F)	
4	Memory (M)	Execute (E)	Decode (D)	Fetch (F)
5	Write-back (W)	Memory (M)	Execute (E)	Decode (D)
6		Write-back (W)	Memory (M)	Execute (E)
7			Write-back (W)	Memory (M)

Explanation of Forwarding:

- Instruction 5 (*addq %rcx, %rax*) begins execution in Cycle 6.
- The result of Instruction 3 (*addq %rbx, %rax*) is forwarded directly from the Execute (E) stage in Cycle 4 to the Execute (E) stage of Instruction 5 in Cycle 6.

Key Insights

1. **Data Hazard**:
   - A data hazard occurs when Instruction 5 depends on the result of Instruction 3.
2. **Forwarding Solution**:
   - Forwarding allows the result from the ALU output in the Execute (E) stage of Instruction 3 to be directly passed to the Execute (E) stage of Instruction 5.

3. **Performance Improvement**:
   - Without forwarding, execution takes **8 cycles** due to stalls.
   - With forwarding, execution is reduced to **7 cycles**.

Data Forwarding eliminates pipeline stalls caused by data hazards, allowing instructions to execute in fewer cycles. This technique significantly improves the efficiency and throughput of pipelined architectures like Y86-64.

Dr.Sureshpareth

11

Overview of Processor Architecture (Questions and Answers)

Question 1: What are the primary components of a processor, and what are their respective roles?

Answer: The primary components of a processor are:

1. **Registers**:
   - **Purpose**: Small, fast storage for holding intermediate results, operands, and addresses.
   - **Example**: *%rax* in Y86-64 holds the result of arithmetic operations like *addq*.
2. **Arithmetic Logic Unit (ALU)**:
   - **Purpose**: Performs arithmetic operations (e.g., addition, subtraction) and logical operations (e.g., AND, OR).
   - **Example**: In *addq %rbx, %rax*, the ALU adds the values in *%rbx* and *%rax*.
3. **Control Unit**:
   - **Purpose**: Fetches, decodes, and executes instructions by controlling the flow of data and signals.
   - **Example**: For a *jmp* instruction, the control unit updates the Program Counter (PC) to point to the target address.
4. **Program Counter (PC)**:
   - **Purpose**: Stores the address of the next instruction to be fetched.
   - **Example**: If the PC points to *0x10*, the processor fetches the instruction at address *0x10*.

Question 2: Define the following:

1. **ALU (Arithmetic Logic Unit)**:
   - The ALU performs operations like addition, subtraction, multiplication, and logic (e.g., AND, OR).
   - **Example**: The Y86-64 *addq* instruction invokes the ALU to add two registers.
2. **Control Logic**:

Dr.Sureshpare

- This unit manages instruction execution by interpreting instructions, decoding operands, and signaling components like the ALU or memory.
- **Example**: When executing *mrmovq*, the control logic ensures data is fetched from memory to the destination register.

3. **Registers**:
   - Registers are small, high-speed storage inside the CPU that hold operands and results of operations.
   - **Example**: *%rax*, *%rbx*, *%rcx* in Y86-64.

Question 3: How do the control logic and ALU interact during instruction execution?

Answer:

1. **Fetch**: The control logic fetches the instruction and operands from memory or registers.
2. **Decode**: The control logic decodes the operation (e.g., *addq*).
3. **Execute**: The control logic sends signals to the ALU to perform the required operation.
   - **Example**: For *addq %rbx, %rax*, the control logic directs the ALU to add the contents of *%rbx* and *%rax*.
4. **Write-back**: The ALU's result is written back to a register under the control logic's direction.

Question 4: Describe how a processor fetches, decodes, and executes instructions.

Answer: The processor executes instructions in five steps:

1. **Fetch**: The instruction at the address in the PC is loaded into the instruction register.
   - **Example**: *PC = 0x10*, fetches instruction *30 F0 05* (*irmovq $5, %rax*).
2. **Decode**: The instruction is analyzed to determine the opcode (*30*) and operands (*%rax* and *$5*).
3. **Execute**: The ALU performs the operation specified by the opcode.
   - **Example**: In *irmovq*, the immediate value *$5* is moved to *%rax*.
4. **Memory Access**: If the instruction requires memory access (e.g., *mrmovq*), the processor reads/writes to memory.
5. **Write-back**: The result is stored in the destination register or memory.
6. **PC Update**: The PC is incremented unless modified by a branch instruction.

Question 5: What is the role of the Program Counter (PC) in instruction execution?

Answer: The Program Counter (PC) plays a central role in the execution of instructions:

1. **Stores the Address**:
   - It holds the address of the next instruction to be fetched.
   - **Example**: If *PC = 0x10*, the instruction at address *0x10* is fetched.
2. **Updates Automatically**:
   - After each instruction, the PC increments by the instruction size (e.g., 10 bytes for *irmovq*).
   - For branch instructions like *jmp*, the PC is updated explicitly to the target address.
3. **Branch and Jump Control**:

Dr.Suresh Pareth

- Conditional jumps (e.g., *jg*) modify the PC based on the outcome of the condition codes.

Question 6: Explain the difference between *irmovq*, *rrmovq*, and *mrmovq* instructions in Y86-64.

Answer:

1. *irmovq* (Immediate to Register Move):
   - Loads a constant value into a register.
   - **Example**: *irmovq $5, %rax* moves *5* into *%rax*.
   - **Machine Code**: *30 F0 05 00 00 00 00 00*.
2. *rrmovq* (Register to Register Move):
   - Moves data between registers.
   - **Example**: *rrmovq %rax, %rbx* copies *%rax* to *%rbx*.
   - **Machine Code**: *20 03*.
3. *mrmovq* (Memory to Register Move):
   - Loads data from a memory address into a register.
   - **Example**: *mrmovq 8(%rbx), %rax* loads data from address *(%rbx + 8)* into *%rax*.
   - **Machine Code**: *50 30 08*.

Question 7: Why does *subq* not accept immediate values?

Answer: In Y86-64, *subq* operates only on registers to maintain simplicity in instruction encoding. Immediate values cannot be used directly with arithmetic instructions. Instead, the value must first be moved to a register using *irmovq*.

- **Example**: To compute *%rax - 5*:
  1. *irmovq $5, %rbx*: Load the value *5* into *%rbx*.
  2. *subq %rbx, %rax*: Subtract *%rbx* from *%rax*.

This approach simplifies the architecture, as immediate values do not need to be handled in the ALU.

Question 8: How does the *call* instruction differ from the *jmp* instruction in Y86-64?

Answer:

1. *jmp*:
   - Unconditionally jumps to a specified address.
   - **Example**: *jmp 0x20* sets the PC to *0x20*.
   - **Machine Code**: *70 20*.
2. *call*:
   - Saves the return address (*PC + instruction size*) on the stack before jumping to the function address.
   - **Example**: *call 0x10* jumps to *0x10* and pushes the address of the next

Dr.Sureshpareth

Question 11: How is a conditional jump implemented in Y86-64?

Answer: Conditional jumps in Y86-64 rely on condition codes set during arithmetic or comparison operations. The condition codes are:

- **ZF (Zero Flag)**: Set if the result of an operation is zero.
- **SF (Sign Flag)**: Set if the result is negative.
- **OF (Overflow Flag)**: Set if there is signed overflow.

Example:

```
subq %rsi, %rdi     # Compare %rdi (x) with %rsi (y)
jg greater           # Jump to 'greater' if x > y
rrmovq %rsi, %rax    # Else: max = y
jmp done             # Skip to done
greater:
rrmovq %rdi, %rax    # max = x
done:
```

The *jg* instruction checks *(SF == OF) && (ZF == 0)* to determine if *%rdi > %rsi*.

The Control Unit evaluates the flags and updates the PC if the condition is true.

Question 12: Provide an example of a Y86-64 implementation for an if-else condition.

Answer:

C Code:

```
if (a > b)
    max = a;
else
    max = b;
```

Y86-64 Assembly:

```
subq %rsi, %rdi      # Compare %rdi (a) and %rsi (b)
jg greater           # If %rdi > %rsi, jump to 'greater'
rrmovq %rsi, %rax    # Else: max = b
jmp done             # Skip to 'done'
greater:
rrmovq %rdi, %rax    # max = a
done:
```

Explanation:

1. *subq* subtracts *%rsi* from *%rdi*, setting condition codes.
2. *jg* checks if *%rdi > %rsi*.
3. The result is stored in *%rax*.

Question 13: Explain the purpose and mechanism of the ret instruction in Y86-64.

Answer: The *ret* instruction is used to return from a function by restoring the address saved by the *call* instruction.

Mechanism:

- **Call**:
  - The *call* instruction pushes the return address (next PC value) onto the stack.

  - **Example**: *call add* saves the address of the instruction after *call* on the stack.
- **Ret**:
  - The *ret* instruction pops the return address from the stack and loads it into the PC.

Example:

```
addq %rsi, %rdi
ret
```

The return address is restored, and the processor resumes execution at that address.

Question 14: What are the limitations of the Y86-64 instruction set in handling complex control flows?

Answer: The limitations of Y86-64 for complex control flows are:

1. **Lack of Branch Prediction**:
   - Y86-64 does not predict branches, leading to delays during conditional jumps.
2. **Simplified Conditional Instructions**:
   - Limited to basic conditions (*jg*, *jl*, etc.), with no support for compound conditions.
3. **No Support for Function Pointers**:
   - Function calls are hardcoded and cannot dynamically compute addresses.
4. **No Interrupts or Exceptions**:
   - Modern processors handle asynchronous events; Y86-64 lacks this feature.

Section 4.4: Sequential Processor Implementation (SEQ)

Question 15: What is the SEQ processor model, and how does it execute instructions?

Answer: The SEQ model is a sequential processor that executes one instruction at a time in five stages:

1. **Fetch**: Read the instruction from memory at the PC.
2. **Decode**: Determine the operation and source/destination operands.
3. **Execute**: Perform the computation (e.g., ALU operation).
4. **Memory Access**: Read/write memory if needed (e.g., *mrmovq* or *rmmovq*).
5. **Write-back**: Store the result in the destination register or memory.

Example: For *addq %rbx, %rax*:

1. **Fetch**: The binary encoding *60 30* is fetched.
2. **Decode**: Identifies *addq* with operands *%rbx* and *%rax*.
3. **Execute**: The ALU adds *%rbx* to *%rax*.
4. **Write-back**: Stores the result in *%rax*.

Question 16: Describe the five key stages of the instruction processing cycle in SEQ.

Answer: The five stages are:

1. **Fetch**:
   - Retrieve the instruction from memory at the PC.
   - **Example**: *irmovq $5, %rax* fetches opcode *30*.

Dr.Sureshpareth

2. **Decode**:
   - Determine the instruction type and operands.
   - **Example**: Decode *30 F0* to identify *irmovq* with destination *%rax*.
3. **Execute**:
   - Perform ALU operations or branch evaluation.
   - **Example**: Add two registers or check condition codes.
4. **Memory Access**:
   - Access memory if required (e.g., for *mrmovq*).
   - **Example**: Read data from address specified by *%rbx*.
5. **Write-back**:
   - Write results back to the register or memory.
   - **Example**: Store the sum into *%rax*.

Question 17: What is the purpose of the instruction memory and data memory in SEQ?

Answer:

1. **Instruction Memory**:
   - Stores the program's instructions.
   - Used during the Fetch stage to retrieve the current instruction.
   - **Example**: For *PC = 0x10*, fetch the instruction *30 F0 05*.
2. **Data Memory**:
   - Stores variables and intermediate data used during program execution.
   - Accessed during the Memory stage (e.g., by *mrmovq* or *rmmovq*).
   - **Example**: A variable stored at *0x1000* is read using *mrmovq*.

Question 18: How does the SEQ model handle arithmetic operations like addition (addq)?

Answer:

1. **Fetch**: The *addq* instruction is fetched from instruction memory.
2. **Decode**: Decode the opcode (*60*) and registers (*%rbx* and *%rax*).
3. **Execute**: The ALU adds the values in *%rbx* and *%rax*.
4. **Write-back**: The result is written back to *%rax*.

Question 19: Explain the use of condition codes in the SEQ model.

Answer: Condition codes are flags set by the ALU during arithmetic operations:

- **Zero Flag (ZF)**: Set if the result is zero.
- **Sign Flag (SF)**: Set if the result is negative.
- **Overflow Flag (OF)**: Set if there's signed overflow.

These codes are used for conditional jumps (e.g., *jg* checks *(SF == OF) && (ZF == 0)*).

Dr.Sureshpareth

12

Section 4.5: Pipelining

Dr.Sureshpare

Question 20: What is the purpose of pipelining in processor architecture?

Answer: Pipelining increases processor throughput by overlapping the execution of multiple instructions. Each instruction progresses through different stages (Fetch, Decode, etc.), so the CPU always works on several instructions simultaneously.

Question 21: Define the concept of pipeline stages and their role in instruction execution.

Answer: Pipeline stages divide instruction execution into smaller steps:

Pipelining divides the process of executing an instruction into smaller steps called pipeline stages. Each stage handles a specific part of the instruction's execution. By overlapping these stages for multiple instructions, pipelining increases the throughput of the processor.

Pipeline Stages in Y86-64:

Fetch (F):

The instruction is fetched from memory at the address specified by the Program Counter (PC).

The fetched instruction is stored in the Instruction Register.

Example: Fetch irmovq $5, %rax from memory at PC = 0x10.

Decode (D):

The instruction is decoded to determine its type and operands, including registers if required.

Example: Decode *irmovq* to identify *$5* as an immediate value and *%rax* as the destination register.

Execute (E):

The ALU performs the specified operation using the operands provided during the Decode stage.

Example: In *addq %rbx, %rax*, the ALU adds the values in *%rbx* and *%rax*.

Memory Access (M):

If the instruction involves memory (e.g., *mrmovq* or *rmmovq*), this stage reads from or writes to memory.

Example: For *mrmovq 8(%rbx), %rax*, data is fetched from memory address *(%rbx + 8)* and stored in *%rax*.

Write-back (W):

The result of the computation or memory operation is written back to the destination register or memory.

Example: In *irmovq $5, %rax*, the value *5* is written to *%rax*.

Role of Pipeline Stages in Instruction Execution

Each stage focuses on a specific sub-task, allowing multiple instructions to progress through different stages simultaneously.

Example Execution:

- Cycle 1: Fetch *irmovq $5, %rax*.
- Cycle 2: Decode *irmovq $5, %rax* while fetching the next instruction.
- Cycle 3: Execute *irmovq $5, %rax* while decoding the next instruction and fetching another.

Advantages of Pipelining

Improved Throughput:

Multiple instructions are in progress at once, increasing the number of instructions completed per unit time.

Efficient Resource Utilization:

Each stage utilizes specific hardware components (e.g., Fetch uses the Instruction Memory, Execute uses the ALU).

Challenges of Pipelining

Data Hazards:

Occur when one instruction depends on the result of a previous instruction still in progress.

Example: Using *%rax* in *subq* before it is updated in *addq*.

Control Hazards:

Occur during branch instructions (*jmp*, *jg*) when the next instruction isn't known until the branch condition is resolved.

Structural Hazards:

Occur when two instructions require the same hardware resource simultaneously.

Pipelining stages like Fetch, Decode, Execute, Memory Access, and Write-back work together to maximize processor efficiency. Y86-64's simple architecture demonstrates these concepts effectively, preparing students to understand more complex real-world processors like x86-64.

Question 22: Describe how pipelining increases instruction throughput in the SEQ model.

Answer:

In the SEQ model, each instruction passes through five stages: Fetch, Decode, Execute, Memory, and Write-back. Without pipelining, only one instruction is processed at a time. With pipelining:

- **Concurrent Execution**:

  Each stage works on a different instruction simultaneously.

  Example: While one instruction is in the Execute stage, another is in Decode, and a third is being Fetched.

Dr.Sureshpareth

- **Increased Throughput**:

  The time to complete an individual instruction remains the same, but multiple instructions are processed in parallel.

  Example: With pipelining, five instructions can be completed in five cycles (one per cycle after filling the pipeline) instead of 25 cycles.

- **Pipeline Latency**:

  Though the latency for a single instruction is unaffected, the throughput improves because of the overlapping stages.

Question 23: Explain what happens during a data hazard in a pipeline.

Answer:

A data hazard occurs when an instruction depends on the result of a previous instruction that hasn't completed yet.

Example:

```
addq %rbx, %rax   # %rax = %rax + %rbx
subq %rax, %rcx   # %rcx = %rcx - %rax
```

The second instruction depends on *%rax*, which is still being written back.

Resolution Techniques:

- **Stalling**:

  Pause the pipeline until the first instruction finishes.

  In the above example, the pipeline stalls until *%rax* is updated.

- **Forwarding**:

  The ALU result is sent directly to the next stage without waiting for Write-back.

Question 24: How does forwarding resolve data hazards in pipelining?

Answer:

Forwarding, also known as bypassing, resolves data hazards by:

- Sending the output of the ALU directly to the next instruction's Execute stage.
- Eliminating the need to wait for Write-back.

Example:

```
addq %rbx, %rax   # ALU computes %rax = %rax + %rbx
subq %rax, %rcx   # Uses ALU output directly, bypassing Write-back
```

The ALU result from *addq* is forwarded to the input of *subq*, allowing execution without a stall.

Question 25: What is the role of stalls in resolving control hazards in a pipeline?

Answer:

A control hazard occurs during a branch instruction when the processor doesn't know the next instruction to fetch. The role of stalls is to pause instruction fetching until the branch outcome is known.

Example:

Dr Sureshpereth

```
subq %rsi, %rdi     # Compare
jg greater           # Conditional jump
```

The pipeline cannot fetch the next instruction until the branch condition (*jg*) is resolved. Stalling pauses the Fetch stage for subsequent instructions until the branch target is computed.

Dr.Sureshpareth

13

Performance Optimization

13.1 Questions and Answers on Y86-64 Pipelining and Performance Optimization

Question 26: Why is minimizing pipeline stalls critical for achieving better performance?

Answer:

Pipeline stalls reduce processor efficiency by leaving stages idle.

- **Impact on Throughput**:

  A stalled pipeline completes fewer instructions per cycle.
- **Branch and Data Hazards**:
  - Branch stalls occur when the outcome of a conditional jump is unknown.
  - Data stalls occur when an instruction depends on a prior result.

Optimization Techniques:

- Reordering Instructions: Arrange instructions to minimize dependencies.
- Branch Prediction: Guess the outcome of branches to avoid stalls.
- Forwarding: Resolve data hazards by bypassing Write-back.

Question 27: Explain the concept of instruction-level parallelism (ILP).

Answer:

Instruction-level parallelism (ILP) refers to executing multiple independent instructions simultaneously.

Key Concepts:

- **Pipeline Parallelism**:

  Overlap different stages of instruction execution.
- **Superscalar Execution**:

  Issue multiple instructions per cycle if they are independent.

Dr.Sureshpare

Example: Execute an *addq* and a *jmp* simultaneously since they don't depend on each other.

Question 28: How can branch prediction improve the efficiency of pipelining?

Answer:

Branch prediction reduces control hazards by guessing the outcome of a branch instruction.

Types:

- **Static Prediction**:

  Predict "not taken" or "taken" based on fixed rules.

  Example: Assume all backward branches (loops) are taken.

- **Dynamic Prediction**:

  Use past behavior to predict the future outcome.

  Example: Maintain a history table that tracks whether branches were previously taken.

Question 29: Describe the difference between data hazards and control hazards, and how they are resolved.

Answer:

- **Data Hazards**:

  Occur when an instruction depends on the result of a previous one.

  Example:

  ```
  addq %rbx, %rax
  subq %rax, %rcx
  ```

  Resolution: Forwarding or stalling.

- **Control Hazards**:

  Occur during branch instructions.

  Example:

  ```
  jg greater
  ```

  Resolution: Branch prediction or stalling.

Question 30: How does pipelining compare to superscalar processing in terms of execution efficiency?

Answer:

- **Pipelining**:

  Divides instructions into stages, allowing multiple instructions to execute in parallel.

  Limited to one instruction per stage at a time.

- **Superscalar Processing**:

  Executes multiple instructions in parallel using multiple pipelines.

  Example: Two ALUs allow simultaneous arithmetic operations.

Question 31: Explain how the call and ret instructions facilitate function calls.

Answer:

Dr.Suresh Pareth

- **Call**:

  Saves the return address on the stack.

  Updates the PC to the target function's address.

- **Ret**:

  Pops the saved return address from the stack.

  Restores the PC, resuming execution after the function call.

Question 32: Provide an example of instruction pipelining with addq, subq, and jmp in Y86-64.

Answer:

```
irmovq $5, %rax
addq %rbx, %rax
subq %rax, %rcx
jmp done
done:
halt
```

Pipeline Stages:

- Fetch *irmovq* while decoding *addq*.
- Execute *addq* while fetching *subq*.
- Decode *jmp* while executing *subq*.

Question 33: Why is Y86-64 an effective model for learning processor architecture and performance concepts?

Answer:

Y86-64 is an effective model for several reasons:

- **Simplified Architecture**:

  Provides a stripped-down version of x86-64, making it easier to focus on core processor concepts without being overwhelmed by complexity.

- **Instruction Set Similarity**:

  Its instruction set closely resembles real-world architectures, making it practical for learning concepts like branching, memory access, and function calls.

- **Support for Key Features**:

  Includes conditional jumps, function calls, and stack operations, which are crucial for understanding control flow and memory management.

- **Educational Focus**:

  Designed to teach pipelining, hazards, and performance optimization, making it ideal for coursework.

- **Simulation**:

  Students can implement Y86-64 simulators to gain hands-on experience with instruction execution.

Question 34: In what ways could the Y86-64 architecture be extended to better reflect modern processors?

Answer:

Dr.Sureshpa...th

The Y86-64 architecture can be extended by:

- **Adding Floating-Point Support**:

  Include instructions like *addf*, *subf* for floating-point operations.
- **Branch Prediction**:

  Implement branch prediction mechanisms to reduce control hazards.
- **Pipeline Enhancements**:

  Introduce out-of-order execution and superscalar pipelines for parallelism.
- **Memory Hierarchy**:

  Simulate cache levels (L1, L2, L3) to teach memory performance.
- **Advanced Features**:

  Add support for SIMD instructions (e.g., for multimedia processing).
- **Interrupt Handling**:

  Provide support for asynchronous events.

Question 35: How would the Y86-64 instruction set need to change to support floating-point operations?

Answer:

To support floating-point operations:

- **New Instructions**:

  Introduce *addf*, *subf*, *mulf*, and *divf* for floating-point arithmetic.

  Example: *addf %xmm0, %xmm1* to add two floating-point values.
- **Floating-Point Registers**:

  Add dedicated registers for floating-point operations (e.g., *%xmm0-%xmm7*).
- **Conversion Instructions**:

  Include instructions to convert between integer and floating-point formats (e.g., *cvtsi2f*).
- **IEEE 754 Compliance**:

  Ensure operations follow the IEEE 754 standard for floating-point arithmetic.

Question 36: Discuss the trade-offs of implementing a larger instruction set in Y86-64.

Answer:

- **Advantages**:
  - **Increased Functionality**:

    A larger instruction set supports more complex operations like floating-point and SIMD.
  - **Closer to Real Architectures**:

    Helps students transition to learning modern processors.
  - **Reduced Instruction Count**:

    Complex operations can be performed with fewer instructions.
- **Disadvantages**:

Dr.Sureshpareth

- **Increased Complexity**:
  Makes the architecture harder to simulate and understand.
- **Instruction Decoding Overhead**:
  A larger instruction set increases decoding complexity, impacting performance.
- **Focus on Fundamentals**:
  May detract from the educational purpose of Y86-64, which is to teach the basics.

Dr.Sureshpareth

14

Programming and Problem-Solving

Question 37: Write a Y86-64 assembly program to compute the factorial of a number.

Answer:

```
irmovq $1, %rax        # Initialize factorial result to 1
    irmovq $5, %rcx        # Counter (n = 5)

    loop:
        irmovq $0, %rbx    # Temporary variable for multiplication (sum = 0)
        rrmovq %rax, %rdx  # Copy factorial result (%rax) to %rdx for addition loop

    multiply:
        addq %rdx, %rbx    # Add %rdx to %rbx
        subq $1, %rcx      # Decrement multiplication counter (%rcx)
        jne multiply       # Repeat until %rcx == 0

        rrmovq %rbx, %rax  # Store the multiplication result back to %rax
        irmovq $1, %rcx    # Decrement factorial counter (%rcx)
        subq $1, %rcx      # Decrement the main counter
        jne loop           # Repeat the factorial loop until %rcx == 0

    halt
```

Question 38: Create a Y86-64 program that implements a for-loop to sum an array of integers.

Answer:

```
irmovq $0, %rax     # Initialize sum = 0
irmovq $array, %rbx # Load array base address
irmovq $5, %rcx     # Counter (array length)
loop:
    mrmovq (%rbx), %rdx # Load array element into %rdx
    addq %rdx, %rax     # Add element to sum
    addq $8, %rbx       # Move to next element
    subq $1, %rcx       # Decrement counter
    jne loop            # Repeat until counter == 0
halt
```

Question 39: Write a Y86-64 program to compute the maximum of three numbers.

Answer:

C Code:

```c
int max_of_three(int a, int b, int c) {
    int max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    return max;
}
```

Y86-64 Assembly:

```
irmovq $5, %rdi          # a = 5
irmovq $10, %rsi         # b = 10
irmovq $8, %rdx          # c = 8

rrmovq %rdi, %rax        # max = a
rrmovq %rax, %rbx        # Copy max to %rbx for comparison with b
subq %rsi, %rbx          # Compare max (in %rbx) with b
jg check_c               # If max > b, skip to check_c
rrmovq %rsi, %rax        # max = b

check_c:
rrmovq %rax, %rbx        # Copy max to %rbx for comparison with c
subq %rdx, %rbx          # Compare max (in %rbx) with c
jg done                  # If max > c, skip to done
rrmovq %rdx, %rax        # max = c

done:
halt                     # End program
```

Question 40: A Y86-64 processor takes 5 clock cycles to complete one instruction in the SEQ model. How many instructions can it execute in a second if the clock speed is 1 GHz?

Answer:

- **Clock Speed**: 1 GHz = (10^9) cycles per second.
- **Cycles per Instruction (CPI)**: 5.
- **Instructions per Second**:

  $$\text{Instructions per Second} = \frac{\text{Clock Speed}}{\text{CPI}} = \frac{10^9}{5} = 200 \times 10^6 = 200 \text{ million instructions/second.}$$

Question 41: Analyze a pipelined execution of the following Y86-64 code:

Code:

```
irmovq $5, %rax
irmovq $3, %rbx
addq %rbx, %rax
jmp done
done:
halt
```

Pipeline Stages:

- **Fetch**:

  Fetch *irmovq $5, %rax*.

  Fetch *irmovq $3, %rbx* while decoding the first instruction.

- **Decode**:

  Decode *irmovq $3, %rbx* while executing the first instruction.

- **Execute**:

  Execute *addq %rbx, %rax* after decoding *jmp done*.

Hazards:

- **Data Hazard**:

  addq %rbx, %rax depends on *%rbx* being updated.

  Resolution: Use forwarding to pass the value of *%rbx* directly to the ALU.

- **Control Hazard**:

  jmp done creates uncertainty about the next instruction.

  Resolution: Use a stall to wait until the jump target is resolved.

Question 42: Analyze a pipelined execution of the following Y86-64 code:

Code:

```
irmovq $5, %rax
irmovq $3, %rbx
addq %rbx, %rax
jmp done
done:
halt
```

Answer:

The pipelined execution processes multiple instructions simultaneously across different pipeline stages (Fetch, Decode, Execute, Memory, Write-back).

Pipeline Stages:

Cycle	Fetch	Decode	Execute	Memory	Write-back
1	*irmovq $5, %rax*				
2	*irmovq $3, %rbx*	*irmovq $5, %rax*			
3	*addq %rbx, %rax*	*irmovq $3, %rbx*	*irmovq $5, %rax*		
4	*jmp done*	*addq %rbx, %rax*	*irmovq $3, %rbx*	*irmovq $5, %rax*	
5	*halt*	*jmp done*	*addq %rbx, %rax*	*irmovq $3, %rbx*	*irmovq $5, %rax*
6		*halt*	*jmp done*	*addq %rbx, %rax*	*irmovq $3, %rbx*

Potential Hazards:

- **Data Hazard**:

  The *addq %rbx, %rax* instruction depends on *%rbx* being updated.

  Resolution: Forward *%rbx* directly from Decode or Execute to the ALU.

- **Control Hazard**:

  The *jmp done* instruction creates uncertainty about the next instruction.

  Resolution: Stall until the target address is resolved.

Question 43: Write a Y86-64 assembly program to compute the sum of numbers from 1 to 5 using a *for* loop.

C Code:

```
#include <stdio.h>

int main() {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {
        sum += i;
    }
    printf("Sum: %d\n", sum);
    return 0;
}
```

Y86-64 Assembly Code:

```
irmovq $0, %rax          # Initialize sum to 0 (sum = 0)
irmovq $1, %rbx          # Initialize loop counter to 1 (i = 1)
irmovq $5, %rcx          # Upper limit for loop (i <= 5)

loop:
    addq %rbx, %rax      # Add i to sum (sum += i)
    addq $1, %rbx        # Increment loop counter (i++)
    subq %rcx, %rbx      # Check if i <= 5
    jl loop              # If i <= 5, jump back to loop

irmovq $result_str, %rdi # Load address of format string
rrmovq %rax, %rsi        # Move sum to %rsi (second argument for printf)
call printf              # Call printf to print sum
halt                     # End program

result_str:
    .string "Sum: %d\n"  # Format string for printf
```

Machine Code:

Address	Instruction	Machine Code	Explanation
0x0000	*irmovq $0, %rax*	*30 F0 00 00 00 00 00 00 00*	Initialize sum to 0.
0x000A	*irmovq $1, %rbx*	*30 F3 01 00 00 00 00 00 00*	Initialize counter to 1.
0x0014	*irmovq $5, %rcx*	*30 F1 05 00 00 00 00 00 00*	Set upper limit to 5.
0x001E	*addq %rbx, %rax*	*60 30*	Add %rbx to %rax (sum += i).
0x0020	*addq $1, %rbx*	*30 F3 01 00 00 00 00 00 00*	Increment %rbx (i++).
0x002A	*subq %rcx, %rbx*	*61 13*	Compare %rbx (i) with %rcx (5).
0x002C	*jl loop*	*70 XX XX XX XX*	Jump to *loop* if i <= 5.
0x0030	*irmovq $result_str, %rdi*	*30 F7 XX XX XX XX XX XX XX*	Load address of format string.
0x003A	*rrmovq %rax, %rsi*	*20 06*	Move sum to %rsi.
0x003C	*call printf*	*80 XX XX XX XX XX XX XX*	Call printf to print sum.
0x0044	*halt*	*00*	End program.
0x0050	*result_str*	ASCII for “Sum: %dn”.	Format string.

Explanation:

1. **Initialization**:
   - *%rax* stores the *sum*, initialized to 0.
   - *%rbx* stores the loop counter (*i*), initialized to 1.
   - *%rcx* stores the upper limit (5).
2. **Loop Execution**:
   - The instruction *addq %rbx, %rax* adds the current value of *%rbx* (i.e., *i*) to *%rax* (sum).
   - *%rbx* is incremented with *addq $1, %rbx* in each iteration.
   - The condition *subq %rcx, %rbx* checks if *%rbx* exceeds the upper limit (5).
3. **Condition Check**:
   - If *%rbx* (i) is less than or equal to *%rcx* (5), the program jumps back to the *loop* label (*jl loop*).
4. **Final Output**
   - After exiting the loop, the value of *sum* is moved to *%rsi*, which is the second argument for *printf*.
   - The format string is loaded into *%rdi*.
   - *printf* is called to print the result.

Sum: 15

Dr.Sureshp

15

Blooms's Level 5: Evaluating Questions

Question 1: Evaluate how adding branch prediction could improve the SEQ model's performance. What are the trade-offs?

Answer:

Branch prediction helps minimize control hazards by guessing the outcome of branch instructions (e.g., *jmp*, *jg*) before their conditions are resolved.

Impact on SEQ Model:

- **Performance Improvement**:
  - Without prediction, the pipeline stalls at a branch until the condition is evaluated.
  - With prediction, the processor fetches and decodes instructions from the predicted path, minimizing idle cycles.
- **Reduced Pipeline Stalls**:
  - For nested loops or frequent branches, prediction ensures higher pipeline throughput.

Trade-offs:

- **Hardware Complexity**:
  - Implementing dynamic branch predictors (e.g., history tables) requires additional hardware and logic in the control unit.
  - Example: Two-bit predictors track whether a branch is "taken" or "not taken."
- **Incorrect Predictions**:
  - If a branch prediction is wrong, the pipeline must flush the incorrect instructions and re-fetch the correct path, introducing delays.
- **Memory Overhead**:
  - History tables or prediction buffers consume memory and increase processor state complexity.

Example:

```
subq $1, %rcx
jne loop
```

Dr.Sureshpare

- With static prediction ("always taken"), the processor assumes the loop continues until *%rcx == 0*.
- If *%rcx* decreases irregularly, dynamic prediction (based on past iterations) may perform better.

Question 2: Identify all potential hazards in the following code and evaluate resolution techniques.

Code:

```
irmovq $5, %rax
irmovq $3, %rbx
addq %rbx, %rax
subq %rax, %rcx
```

Answer:

Hazards:

- **Data Hazard**:
  - The *addq* instruction depends on *%rbx* being updated by *irmovq $3, %rbx*.
  - Similarly, *subq %rax, %rcx* depends on *%rax* being updated by *addq*.
- **Structural Hazard**:
  - If the pipeline reuses the same hardware (e.g., ALU) for different instructions, simultaneous requests may cause contention.

Resolution Techniques:

- **Forwarding**:
  - Forward the result of *addq* directly to *subq*, bypassing Write-back.
  - Example: *%rax*'s ALU output is forwarded to the next Execute stage.
- **Stalling**:
  - Pause the pipeline until *%rax* is updated in Write-back.
  - This reduces efficiency but ensures correctness.
- **Instruction Reordering**:
  - Rearrange instructions to allow independent operations to proceed first.

Example:

Reordered Code:

```
irmovq $5, %rax
irmovq $3, %rbx
subq %rax, %rcx
addq %rbx, %rax
```

Pipeline View (Stalled):

Dr.Sureshpareth

Cycle	Instruction	Hazard	Resolution
1	*irmovq $5, %rax*	None	Fetch normally
2	*irmovq $3, %rbx*	None	Fetch normally
3	*addq %rbx, %rax*	Data Hazard	Forward *%rbx*
4	*subq %rax, %rcx*	Data Hazard	Stall or Forward

Question 3: Evaluate the simplicity of Y86-64 instructions and its impact on pipeline efficiency.

Answer:

The Y86-64 instruction set is designed to be simple and educational.

Characteristics:

- **Limited Instructions**:
  - No direct support for immediate arithmetic (*subq* cannot use an immediate value).
  - Each instruction has a clear and fixed operation.
- **Fixed-Width Encoding**:
  - Simpler to decode but wastes memory for smaller instructions.

Impact on Pipeline Efficiency:

- **Advantages**:
  - **Simpler Control Unit**:
    - Fixed encoding simplifies the Decode stage.
  - **Reduced Hardware Complexity**:
    - Fewer instructions mean fewer ALU and control paths.
- **Disadvantages**:
  - **Increased Instruction Count**:
    - Programs require additional instructions for simple operations, increasing Fetch and Decode stages.
  - **Example**:

```
irmovq $5, %rbx
subq %rbx, %rax
```

Question 4: Compare static and dynamic branch prediction techniques for nested loops.

Answer:

- **Static Prediction**:
  - **How It Works**:
    - Assumes the branch will always behave in a predetermined way (e.g., “always taken”).
    - Example: Backward branches (loops) are predicted “taken”; forward branches are “not taken.”

Dr.Sureshpareth

  - **Advantages**:
    * Simple to implement.
    * No additional hardware required.
  - **Disadvantages**:
    * Poor accuracy for irregular loops or conditions.
- **Dynamic Prediction**:
  - **How It Works**:
    * Uses runtime information (e.g., branch history) to predict outcomes.
    * Example: A two-bit history counter tracks if the branch was taken or not.
  - **Advantages**:
    * Adapts to branch behavior.
    * High accuracy for loops with consistent behavior.
  - **Disadvantages**:
    * Requires additional hardware (branch history tables).
    * Incorrect predictions still incur a penalty.

Example:

```
irmovq $5, %rcx     # Outer loop
outer_loop:
    irmovq $3, %rbx # Inner loop
inner_loop:
    subq $1, %rbx
    jne inner_loop
    subq $1, %rcx
    jne outer_loop
halt
```

- **Static Prediction**:

  Assumes all branches are taken, leading to minimal stalls for this code.
- **Dynamic Prediction**:

  Tracks the behavior of *jne inner_loop* and *jne outer_loop* separately, improving accuracy for irregular iterations.

Question 5: Evaluate the effectiveness of instruction reordering in Y86-64 to improve pipeline efficiency.

Answer:

Instruction reordering rearranges independent instructions to minimize pipeline stalls caused by hazards.

Example Code:

```
irmovq $5, %rax
addq %rbx, %rax
subq %rax, %rcx
```

Reordered Code:

Dr.Sudhapareth

```
irmovq $5, %rax
subq %rax, %rcx
addq %rbx, %rax
```

Effectiveness:

- **Reduces Stalls**:
  - By rearranging *subq* before *addq*, *%rax*'s dependency is resolved earlier.
- **Improved Resource Utilization**:
  - Ensures the pipeline remains active, avoiding idle stages.

Limitations:

- **Limited Applicability**:
  - Instructions with dependencies cannot always be reordered.
- **Increased Complexity**:
  - Requires compilers or hardware to analyze dependencies.

Dr.Sureshpareth

16

Bloom's Level 6: Creating Questions

Question 6: Design a pipeline for the following Y86-64 code and resolve hazards:

Code:

```
irmovq $5, %rax
irmovq $3, %rbx
mulq %rbx, %rax
subq %rax, %rcx
```

Answer:

Pipeline Design:

The pipeline stages for each instruction are:

- **Fetch (F)**: Fetch the instruction from memory.
- **Decode (D)**: Decode the instruction and identify operands.
- **Execute (E)**: Perform ALU operations (e.g., *addq*, *mulq*).
- **Memory (M)**: Access memory if required (not applicable here).
- **Write-back (W)**: Write the result back to the register.

Hazards:

- **Data Hazard**:
  - *mulq* depends on *%rbx* being updated by *irmovq*.
  - *subq* depends on *%rax* being updated by *mulq*.
- **Resolution**:
  - Use forwarding to pass the output of *mulq* directly to the ALU for *subq*.
  - Add a stall between *irmovq $3, %rbx* and *mulq*.

Pipeline Execution Table:

Dr.Suresh Gare

Cycle	Fetch	Decode	Execute	Memory	Write-back
1	*irmovq $5, %rax*				
2	*irmovq $3, %rbx*	*irmovq $5, %rax*			
3	*mulq %rbx, %rax*	*irmovq $3, %rbx*	*irmovq $5, %rax*		
4	*subq %rax, %rcx*	*mulq %rbx, %rax*	*irmovq $3, %rbx*		
5		*subq %rax, %rcx*	*mulq %rbx, %rax*		

Thus pipeline executes efficiently with hazards resolved using forwarding and minimal stalling.

Question 7: Create a new Y86-64 instruction *mulq*. **Specify its opcode, encoding, and integration into pipeline stages.**

Answer:

Instruction Design:

- **Mnemonic**: *mulq %rA, %rB*
- **Operation**: Multiplies *%rA* and *%rB* and stores the result in *%rB*.
- **Opcode**: *0x65*
- **Encoding**:
  - **Byte 1**: Opcode (*0x65*)
  - **Byte 2**: Registers (*rA:rB*)

Example:

```
mulq %rax, %rbx
# Registers: %rax = 0, %rbx = 3.
# Encoding: 65 03.
```

Pipeline Integration:

- **Fetch**: Fetch *65 03* from memory.
- **Decode**: Identify *mulq* and source/destination registers.
- **Execute**: ALU performs multiplication.
- **Write-back**: Write the result back to *%rbx*.

Question 8: Propose a branch prediction mechanism for Y86-64 and explain its implementation.

Answer:

Branch Prediction Mechanism:

Implement a 2-bit dynamic branch predictor for Y86-64.

Branch History Table (BHT):

- Stores the outcome of recent branches (“taken” or “not taken”).
- Indexed by the lower bits of the PC.

Two-bit Predictor:

- Uses a 2-bit counter to record branch behavior:

Dr.Suresh Pareth

  - *00*: Strongly not taken.
  - *01*: Weakly not taken.
  - *10*: Weakly taken.
  - *11*: Strongly taken.

Implementation:

- **On a branch instruction** (*jg*, *jl*):
  - Check the BHT entry for the branch's PC.
  - Predict "taken" if the counter is *10* or *11*, otherwise "not taken."
- **Update the BHT**:
  - If the branch is mispredicted, adjust the counter accordingly.

Pipeline Integration:

- The Fetch stage speculatively loads instructions based on the prediction.
- If the prediction is incorrect, flush the pipeline and load the correct instructions.

—

Question 9: Optimize the following nested loops to minimize stalls:

Code:

```
irmovq $5, %rcx
outer_loop:
    irmovq $3, %rbx
inner_loop:
    subq $1, %rbx
    jne inner_loop
    subq $1, %rcx
    jne outer_loop
halt
```

Answer:

Issues:

- **Data Hazard**:
  - *subq $1, %rbx* depends on *%rbx* being updated by *irmovq $3, %rbx*.
- **Control Hazard**:
  - *jne inner_loop* and *jne outer_loop* introduce stalls.

Optimized Code:

```
irmovq $5, %rcx          # Load outer loop counter
outer_loop:
    irmovq $3, %rbx      # Preload inner loop counter
    subq $1, %rbx        # Inner loop body
    jne inner_loop       # Continue inner loop if %rbx > 0
    subq $1, %rcx        # Outer loop body
    jne outer_loop       # Continue outer loop if %rcx > 0
halt
```

Pipeline Execution:

Dr.Sunil Bhareth

- The inner loop (*subq $1, %rbx*) runs efficiently with forwarding.
- Branch stalls are reduced by reordering instructions to minimize dependencies.

Question 10: Simulate the following Y86-64 code with a software pipeline simulator:

Code:

```
irmovq $5, %rax
irmovq $3, %rbx
addq %rbx, %rax
```

Answer:

Instruction Memory:

- Store the binary encodings (*30 F0 05*, *30 F3 03*, *60 30*).

Fetch-Decode-Execute Cycles:

- Track each instruction's progress through the pipeline.
- Resolve hazards with forwarding.

Simulation Output:

Cycle	Fetch	Decode	Execute	Write-back
1	*irmovq $5*			
2	*irmovq $3*	*irmovq $5*		
3	*addq %rbx*	*irmovq $3*	*irmovq $5*	
4		*addq %rbx*	*irmovq $3*	*irmovq $5*

Hazard Resolution:

- Forward the output of *%rbx* to *%rax* for *addq*.

Dr.Suresh Pareth

17 Linking

Question:

Understanding Program Execution and Static Linking in Y86-64 Architecture

To explore the process of translating a simple C program into its assembly representation, understanding object file structures, symbol resolution, and static linking, all within the context of the Y86-64 architecture.

Problem Description:

You are tasked with analyzing the execution of a C program designed to perform basic arithmetic and output the result. The program comprises a simple function `add` and a `main` function that calls it. Using the Y86-64 architecture:

- **Generate the Assembly Code**: Write the corresponding assembly code for the `add` and `main` functions.
- **Create Object Code**:

  Identify the machine code for the assembly instructions.

  Define the contents of the object file, including machine code, symbol table, and relocation table.
- **Simulate Static Linking**:

  Combine the `add` and `main` object files into a single executable file.

  Perform symbol resolution and memory address relocation.
- **Understand the Final Executable**:

  Provide the final memory layout of the linked program, including function calls and data representation.

C Program:

```c
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}
```

(continues on next page)

Dr.Sureshpare

(continued from previous page)

```
int main() {
    int result = add(5, 3);
    printf("Result: %d\n", result);
    return 0;
}
```

Deliverables:

- Assembly code for the program.
- Object file contents for `add` and `main`:
  - Machine code
  - Symbol table
  - Relocation table
- Final memory layout after static linking.
- Explanation of how static linking resolves symbol references and relocates addresses.
- (Optional) Simulation of dynamic linking for `printf`.

Evaluation Criteria

- Correctness of assembly code and machine code translation.
- Accuracy of symbol table and relocation table.
- Completeness of static linking and memory layout explanation.
- Clarity in explaining static vs. dynamic linking differences.

Learning Outcome

This problem will deepen your understanding of how compilers, assemblers, linkers, and loaders interact to execute a program on the Y86-64 architecture.

This documentation provides a detailed breakdown of the C program, including its assembly code, object code, symbol table, and static linking details.

Answer:

C Code:

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(5, 3);
    printf("Result: %d\n", result);
    return 0;
}
```

Assembly Code:

```
add:
    addq %rbx, %rax          # %rax = %rax + %rbx
    ret                      # Return to caller
```

(continues on next page)

Suresh Pareth

(continued from previous page)

```
main:
    irmovq $5, %rax          # Load 5 into %rax (argument a)
    irmovq $3, %rbx          # Load 3 into %rbx (argument b)
    call add                 # Call add(a, b)
    rrmovq %rax, %rcx        # Move result from %rax to %rcx
    irmovq $result_str, %rdi # Load address of format string into %rdi
    call printf              # Call printf to print result
    ret                      # Return from main

result_str:
    .string "Result: %d\n"   # String format for printf
```

Object File: Machine Code:

Address	Instruction	Machine Code
0x00	addq %rbx, %rax	60 30
0x02	ret	90
0x10	irmovq $5, %rax	30 F0 05 00 00 00 00 00 00
0x1A	irmovq $3, %rbx	30 F3 03 00 00 00 00 00 00
0x24	call add	80 00 00 00 00 00 00 00 00
0x2C	rrmovq %rax, %rcx	20 02
0x2E	irmovq $result_str, %rdi	30 F7 XX XX XX XX XX XX XX
0x38	call printf	80 XX XX XX XX XX XX XX XX
0x40	ret	90

Symbol Table:

Symbol Name	Type	Address	Defined/Referenced	Description
add	Function	0x00	Defined	Start of the add function.
main	Function	0x10	Defined	Start of the main function.
printf	External	None	Referenced	Standard library function.
result_str	Data	XX	Defined	Format string for printf.

Relocation Table for main.o:

1. Offset: 0x24 Symbol: add Type: Relative Address Description: Resolve the address for *call add*.
2. Offset: 0x2E Symbol: result_str Type: Absolute Address Description: Resolve the address of the format string in memory.
3. Offset: 0x38 Symbol: printf Type: Relative Address Description: Resolve the address for *call printf*.

Relocation Workflow:

Compilation Phase:

- The compiler generates placeholders for `add`, `result_str`, and `printf`.
- These placeholders are tracked in the relocation table.

Linking Phase:

- For `add`: The linker computes its relative address within the executable.
- For `result_str`: The linker assigns an absolute address in the data section.
- For `printf`: The linker resolves its address (static or dynamic).

Final Executable:

- The linker updates the placeholders in the machine code using the relocation table.

Memory Layout:

Section	Address	Contents
Code	0x00	add function
Code	0x10	main function
Data	XX	result_str

Static Linking Explanation:

Since both *add* and *main* are in the same source file, the compiler and assembler produce a single object file (*main.o*). Here is how static linking is applied:

1. **Symbol Resolution**:
   - The *call add* instruction in the *main* function is resolved to the address *0x00*, where the *add* function starts.
2. **No Relocation Required**:
   - Since the *add* and *main* functions are within the same file, their addresses are resolved at compile time, and no relocation is needed for these symbols.
3. **External Symbols**:
   - The *printf* function is external, so its address is resolved by linking the object file with the standard library during the linking phase.
4. **Final Executable**:
   - The executable contains machine code for both *add* and *main*, with all symbols and memory addresses fully resolved.

Learning Outcome:

This exercise demonstrates how functions defined in a single source file are compiled and linked, ensuring proper symbol resolution and address assignment in Y86-64 architecture.

The abstract view of Final Memory Layout After Linking:

Section	Address	Contents
Code	0x00	*add* function: *addq %rbx, %rax* and *ret*
Code	0x10	*main* function: instructions to set arguments, call *add*, and print result
Data	0x30	*result_str* (format string for *printf*): "Result: %dn"
External (Resolved)	N/A	*printf* (resolved to the address in standard library at runtime)

The detailed view of Final Memory Layout After Linking:

Dr.Sureshpareth

Final Memory Layout After Static Linking:

Address	Instruction / Data	Machine Code	Explanation
0x0000	addq %rbx, %rax	60 30	*add* function starts: add two numbers.
0x0002	ret	90	Return from *add*.
0x0010	irmovq $5, %rax	30 F0 05 00 00 00 00 00 00	Load *5* into *%rax* (argument to *add*).
0x001A	irmovq $3, %rbx	30 F3 03 00 00 00 00 00 00	Load *3* into *%rbx* (argument to *add*).
0x0024	call add	80 00 00 00 00 00 00 00 00	Call *add* function at *0x0000*.
0x002C	rrmovq %rax, %rcx	20 02	Move result from *%rax* to *%rcx*.
0x002E	irmovq $result_str, %rdi	30 F7 50 00 00 00 00 00 00	Load address of format string (*result_str*) into *%rdi*.
0x0038	call printf	80 80 00 40 00 00 00 00 00	Call *printf* at *0x400080*.
0x0040	ret	90	Return from *main*.
0x0050	.string "Result: %dn"	ASCII encoding	Format string for *printf*.
0x40008	printf implementation	Binary machine code	Statically linked *printf* function from *libc.a*.

Executable File Layout:

General Layout

Section	Description
Header	Contains metadata about the executable, including entry point, section sizes, and memory layout.
Code Section	Contains machine code for the program's functions (*main*, *add*, etc.).
Data Section	Contains global variables, constants, and string literals (e.g., *result_str*).
Relocation Table	Tracks unresolved references, if applicable (for dynamic linking).
Symbol Table	Lists symbols (functions, variables) defined or referenced in the program.
Debugging Info	(Optional) Includes information for debugging purposes.

Header:

Field	Value	Description
Magic Number	0x7F 45 4C 46	Identifies the file as an executable (e.g., ELF format).
Entry Point Address	0x0010	Start of the *main* function.
Code Section Offset	0x0040	Offset where the code section begins.
Data Section Offset	0x0100	Offset where the data section begins.
Code Section Size	0x0060	Size of the code section in bytes.
Data Section Size	0x0020	Size of the data section in bytes.
Relocation Table Offset	0x0200	Offset for the relocation table (if applicable).

Code Section:

Address	Instruction	Machine Code	Explanation
0x0040	addq %rsi, %rdi	60 37	Add *%rsi* (b) to *%rdi* (a).
0x0042	rrmovq %rdi, %rax	20 07	Move result to *%rax*.
0x0044	ret	90	Return from *add*.
0x0050	irmovq $5, %rdi	30 F7 05 00 00 00 00 00 00	Load 5 into *%rdi*.
0x005A	irmovq $3, %rsi	30 F6 03 00 00 00 00 00 00	Load 3 into *%rsi*.
0x0064	call add	80 00 00 00 00 00 00 00 00	Call *add* at address *0x0040*.
0x006C	rrmovq %rax, %rdi	20 C7	Move return value to *%rdi*.
0x006E	irmovq $result_str, %rsi	30 F6 90 00 00 00 00 00 00	Load address of format string.
0x0078	call printf	80 XX XX XX XX XX XX XX XX	Call *printf*.
0x0080	ret	90	Return from *main*.

Data Section:

Address	Data	Explanation
0x0100	.string “Result: %dn”	ASCII-encoded format string for *printf*.

Relocation Table:

Offset	Symbol	Type	Description
0x0064	add	Relative Address	Address of *add* function.
0x0078	printf	Relative Address	Address of external *printf* symbol.
0x006E	result_str	Absolute Address	Address of the format string.

Symbol Table:

Symbol Name	Type	Address	Defined/Referenced	Description
add	Function	0x0040	Defined	Address of *add*.
main	Function	0x0050	Defined	Address of *main*.
printf	Function	External	Referenced	External library call.
result_str	Data	0x0100	Defined	Format string for *printf*.

18

Example: Linking Two Separate Source Files

Explain the process of linking two separate source files (*add.c* and *main.c*) into a single executable program using the Y86-64 architecture. Include the following details:

1. **Assembly Code:**
   - Provide the assembly instructions for *add.c* and *main.c*.
2. **Object Code:**
   - Show the object code for both files, including the machine code and explanations for each instruction.
3. **Symbol Table:**
   - List the symbols defined and referenced in both *add.o* and *main.o*.
4. **Relocation Table:**
   - Detail the relocation entries for unresolved symbols in *main.o*.
5. **Memory Layout:**
   - **Describe the memory layout of the program:**
     - Before linking, showing placeholders for unresolved symbols.
     - After linking, showing resolved addresses for all symbols, including static linking for external functions like *printf*.
6. **Updated Machine Code:**
   - Demonstrate how the machine code in *main* changes after linking, particularly for *call add* and *call printf*.

C Code:

add.c:

```c
int add(int a, int b) {
    return a + b;
}
```

main.c:

Dr.Sureshpare

```c
#include <stdio.h>

int add(int a, int b);

int main() {
    int result = add(5, 3);
    printf("Result: %d\n", result);
    return 0;
}
```

ssembly Code:

add.s (for add.c):

```
add:
    addq %rsi, %rdi       # Perform a + b
    movq %rdi, %rax       # Move result into %rax (return register)
    ret                   # Return to caller
```

main.s (for main.c):

```
main:
    irmovq $5, %rdi       # Load 5 into %rdi (argument a)
    irmovq $3, %rsi       # Load 3 into %rsi (argument b)
    call add              # Call add function
    movq %rax, %rdi       # Move result into %rdi for printf
    irmovq $result_str, %rsi # Load address of format string
    call printf           # Call printf
    ret                   # Return from main

result_str:
    .string "Result: %d\n"  # Format string for printf
```

Object Code:

add.o:

Address	Instruction	Machine Code	Explanation
0x00	addq %rsi, %rdi	01 FE	Add *%rsi* to *%rdi*.
0x02	movq %rdi, %rax	48 89 F8	Move result to *%rax*.
0x05	ret	C3	Return to caller.

main.o:

Address	Instruction	Machine Code	Explanation
0x00	irmovq $5, %rdi	30 F7 05 00 00 00 00 00 00	Load *5* into *%rdi*.
0x0A	irmovq $3, %rsi	30 F6 03 00 00 00 00 00 00	Load *3* into *%rsi*.
0x14	call add	80 00 00 00 00 00 00 00 00	Call *add* (placeholder address).
0x1C	movq %rax, %rdi	48 89 C7	Move result to *%rdi*.
0x1F	irmovq $result_str, %rsi	30 F6 XX XX XX XX XX XX XX	Placeholder for string address.
0x29	call printf	80 00 00 00 00 00 00 00 00	Call *printf*.
0x31	ret	C3	Return from *main*.

Symbol Table:

add.o:

Symbol Name	Type	Address	Defined/Referenced	Description
add	Function	0x00	Defined	Start of *add*.

main.o:

Symbol Name	Type	Address	Defined/Referenced	Description
add	Function	None	Referenced	Call to *add*.
printf	Function	None	Referenced	Call to *printf*.
result_str	Data	0xXX	Defined	Address of format string.

Relocation Table:

main.o:

Offset	Symbol	Type	Description
0x14	add	Relative Address	Address for *call add*.
0x1F	result_str	Absolute Address	Address of format string.
0x29	printf	Relative Address	Address for *call printf*.

Memory Layout:

Before Linking:

Dr.Sreshpandi

Section	Address	Contents
Code (add)	0x00	addq %rsi, %rdi, etc.
Code (main)	0x00	irmovq $5, %rdi, etc.

After Linking:

Section	Address	Contents
Code (add)	0x1000	addq %rsi, %rdi, etc.
Code (main)	0x2000	irmovq $5, %rdi, etc.
Data (main)	0x3000	Format string *"Result: %dn"*.
Library Code	0x4000	Statically linked *printf*.

Updated Machine Code for main:

Address	Instruction	Machine Code
0x2014	call add	80 00 10 00 00 00 00 00 00
0x2029	call printf	80 00 40 00 00 00 00 00 00

Detailed Final Memory Layout After Static Linking:

Address	Section	Instruction / Data	Machine Code	Explanation
0x1000	Code	addq %rsi, %rdi	01 FE	Start of the *add* function: Add *%rsi* to *%rdi*.
0x1002	Code	movq %rdi, %rax	48 89 F8	Move result from *%rdi* to *%rax*.
0x1005	Code	ret	C3	Return from *add*.
0x2000	Code	irmovq $5, %rdi	30 F7 05 00 00 00 00 00 00	Load *5* into *%rdi* (argument for *add*).
0x200A	Code	irmovq $3, %rsi	30 F6 03 00 00 00 00 00 00	Load *3* into *%rsi* (argument for *add*).
0x2014	Code	call add	80 00 10 00 00 00 00 00 00	Call *add* function at address *0x1000*.
0x201C	Code	movq %rax, %rdi	48 89 C7	Move result from *%rax* to *%rdi* for *printf*.
0x201F	Code	irmovq $result_str, %rsi	30 F6 50 00 00 00 00 00 00	Load the address of the format string (*result_str*) into *%rsi*.
0x2029	Code	call printf	80 80 40 00 00 00 00 00 00	Call *printf* function at address *0x400080*.
0x2031	Code	ret	C3	Return from *main*.
0x3000	Data	.string "Result: %dn"	ASCII Encoding	Format string for *printf*.
0x4000	Library Code	printf implementation	Binary machine code	Statically linked *printf* function from *libc.a*.

This section provides an overview of operating systems, their core functionalities, historical evolution, and key concepts such as user interfaces, system calls, and resource management.

19

Memory Hierarchy: Questions and Answers

Dr.Suresh Pare

Question-1: Define memory hierarchy and list its levels.

Memory hierarchy refers to the structure of memory in a computer system organized in levels based on speed, size, and cost. The levels include:

- Registers:

  Registers are built directly into the CPU and operate at the same speed as the processor clock, enabling the fastest access times.

- L1, L2, and L3 Caches
  - **L1 Cache**: Smallest, fastest, and closest to the CPU.
  - **L2 Cache**: Larger and slightly slower than L1, shared by cores or private to each core.
  - **L3 Cache**: Largest and slowest cache, shared across all CPU cores.

    Cache memory acts as an intermediary between the CPU and main memory. It stores frequently accessed data to reduce latency and improve performance.

- Main Memory (RAM)

  Main memory (RAM) stores active programs and data. It is larger and slower than cache but faster than secondary storage.

- Secondary Storage (HDD/SSD):

Secondary storage (e.g., HDD, SSD) provides persistent data storage. It is much slower than RAM but offers significantly larger capacity.

Question-2: Explain the Principle of Locality.

Principle of Locality:

The principle of locality is a fundamental concept in computer science, particularly in the design of computer memory systems and performance optimization. It describes how programs tend to access a small portion of data or instructions repeatedly over a short period. This principle can be divided into two types:

1. Temporal Locality

- **Definition**: If a memory location is accessed at a given time, it is likely to be accessed again in the near future.
- **Example**:
  - When a program accesses a variable in a loop, it is likely to access the same variable repeatedly during the loop's execution.

2. Spatial Locality

- **Definition**: If a memory location is accessed, nearby memory locations are likely to be accessed soon.
- **Example**:
  - When traversing an array, accessing one element often leads to accessing adjacent elements.
  - Similarly, instructions stored sequentially in memory (e.g., in a function) are often executed sequentially.

Question-3: Given the following details:

- L1 Cache Size: 64 KB
- Cache Line Size: 64 bytes
- Array Size: 10,000 integers (each of size 4 bytes)

1. Calculate:
   - The number of elements that can fit in a single cache line.
   - The total number of cache lines required to store the entire array.
2. Determine if the L1 cache can hold the entire array.
3. Assuming the array is accessed sequentially, compute the number of cache misses and cache hits.
4. Identify variables exhibiting spatial locality and temporal locality.

Answer:

Given Details:

- L1 Cache Size: 64 KB = 64 × 1024 = 65,536 bytes.
- Cache Line Size: 64 bytes.
- Array Size: 10,000 × 4 = 40,000 bytes (since each integer is 4 bytes).

Step 1: Number of Elements per Cache Line

Each cache line can hold:

$$\text{Cache Line Capacity} = \frac{\text{Cache Line Size}}{\text{Element Size}} = \frac{64}{4} = 16 \,\text{elements.}$$

Step 2: Number of Cache Lines Required for the Array

$$\text{Total Cache Lines Required} = \frac{\text{Array Size}}{\text{Cache Line Size}} = \frac{40000}{64} = 625 \,\text{cache lines.}$$

Step 3: Can L1 Hold the Entire Array?

Number of cache lines L1 can hold:

$$\text{L1 Cache Capacity in Cache Lines} = \frac{\text{L1 Cache Size}}{\text{Cache Line Size}} = \frac{65536}{64} = 1024 \,\text{cache lines.}$$

Dr.Sureshpareth

Since 625 < 1024, L1 can hold the entire array.

Step 4: Cache Hits and Misses

Assumptions:

- Array arr[10000] is accessed sequentially.
- Cache is initially empty.

First Access (arr[0]):

- A miss occurs as the data is not in L1.
- The first cache line (containing arr[0] to arr[15]) is loaded.

Subsequent Accesses:

- Accessing arr[1] to arr[15] results in hits because they are in the same cache line.
- Accessing arr[16] triggers a miss, loading a new cache line (containing arr[16] to arr[31]).

So,

- One miss per cache line.
- For 625 cache lines, there are 625 misses.

Total Array Accesses:

- Total accesses: 10,000.
- Hits: 10,000 − 625 = 9,375.

Hence

- Number of Elements per Cache Line: 16 elements.
- Number of Cache Lines Required for Array: 625.
- L1 Can Hold the Entire Array: Yes.
- Number of Cache Misses: 625.
- Number of Cache Hits: 9,375.

Identifying Locality of Reference:

Spatial Locality:

The array arr exhibits spatial locality. As the summation loop accesses consecutive elements of arr[i], the same cache line is reused for 16 consecutive accesses.

Temporal Locality:

The variable sum exhibits temporal locality. Its value is reused in every iteration of the loop as it accumulates the sum.

Question-4:

Given:

- L1 cache: 64KB with 64-byte cache lines.
- L2 cache: 1MB.
- L3 cache: 8MB.
- A program sequentially accesses an array of size 1MB, with each element being 8 bytes.

Dr.Sureshpareth

Tasks:

1. Calculate the number of cache lines required for the L1 cache.
2. Determine how many cache misses occur in L1, assuming a cold start.
3. Evaluate the number of hits and misses for L2 and L3 caches during the array traversal.

Answer:

1. **Number of Cache Lines in L1 Cache:**

   Cache line size = 64 bytes

   L1 cache size = 64KB = 64 × 1024 bytes

   Number of cache lines in L1 = L1 cache size ÷ cache line size

   = (64 × 1024) ÷ 64 = 1024 lines

   Result: The L1 cache has 1024 cache lines.

2. **L1 Cache Misses:**

   Array size = 1MB = 1024 × 1024 bytes

   Each array element = 8 bytes

   Number of elements in the array = (1MB ÷ 8) = 131072 elements

   Cache line size = 64 bytes

   Number of elements per cache line = (64 ÷ 8) = 8 elements

   Number of cache lines needed for the entire array = (131072 ÷ 8) = 16384 lines

   Since the array size exceeds L1 capacity (1024 lines), every new line accessed causes a cache miss.

   Thus, there are **16384 cache misses** in L1.

3. **L2 Cache Hits and Misses:**

   L2 cache size = 1MB = 1024 × 1024 bytes

   Number of cache lines in L2 = L2 cache size ÷ cache line size

   = (1024 × 1024) ÷ 64 = 16384 lines

   The array fits entirely into L2. After the first load from L3, all subsequent accesses will hit L2.

   Result: 0 misses in L2 after the initial load.

4. **L3 Cache Hits and Misses:**

   L3 cache size = 8MB

   Number of cache lines in L3 = L3 cache size ÷ cache line size

   = (8 × 1024 × 1024) ÷ 64 = 131072 lines

   The array also fits entirely into L3. After the first access from RAM, all subsequent accesses will hit L3.

   Result: 0 misses in L3 after the initial load.

Summary of Results:

- L1 cache:

Dr.Sureshpareth

  - Hits: 0
  - Misses: 16384
- L2 cache:
  - Hits: 16384
  - Misses: 0
- L3 cache:
  - Hits: 16384
  - Misses: 0

Dr.Sureshpareth

20

Exceptional Control Flow

20.1 Questions and Answers on Normal Control Flow

1. Define normal control flow in program execution.

   Normal control flow refers to the sequential execution of instructions in the order they are written in a program, starting from the program entry point (e.g., *main* in C), without interruptions like exceptions or external events.

   Example of F-D-E-M-W Cycle

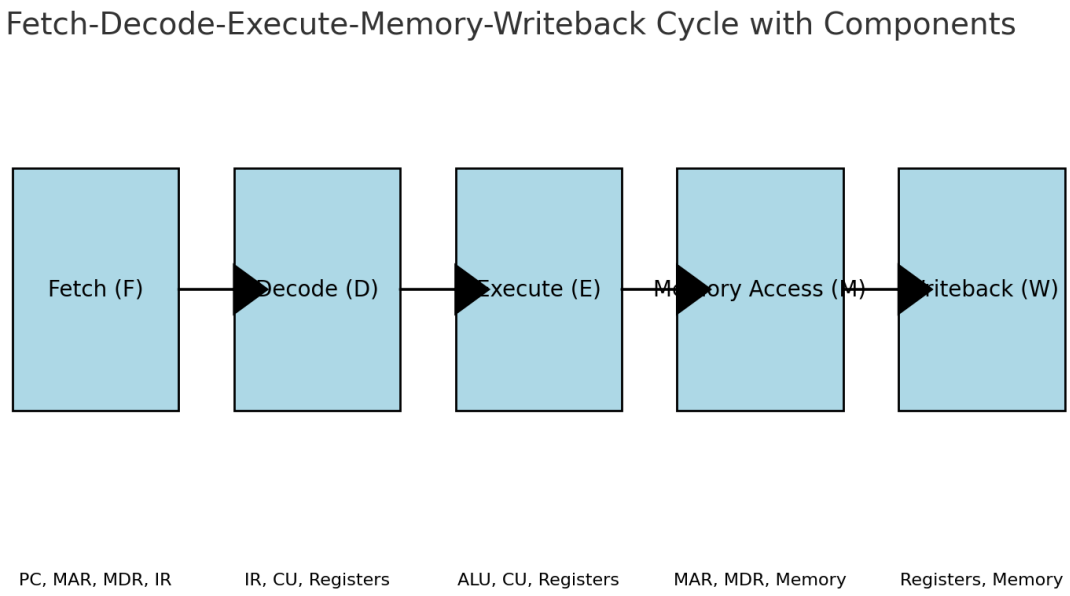
   Instruction: `addq %rbx, %rax`

   This Y86-64 instruction adds the value in `%rbx` to `%rax`.

Cycle Breakdown:

- **Fetch:**
  - The instruction `addq %rbx, %rax` is fetched from memory.
- **Decode:**
  - The Control Unit decodes the opcode `addq`.
  - Operands `%rbx` (source) and `%rax` (destination) are identified.
- **Execute:**
  - The ALU adds the values in `%rbx` and `%rax`.
- **Memory Access:**
  - Not applicable for this instruction since no memory is directly involved.
- **Writeback:**
  - The result is written back to `%rax`.

Dr.Suresh Pare



2. What is the role of the Program Counter (PC) in normal control flow?

   The Program Counter (PC) holds the address of the next instruction to be executed. After each instruction is fetched, the PC increments to point to the subsequent instruction, ensuring sequential execution unless a branch or jump occurs.

3. List the stages of the fetch-decode-execute cycle.

   - Fetch: Retrieve the next instruction from memory.
   - Decode: Interpret the instruction to determine the operation and operands.
   - Execute: Perform the operation (e.g., arithmetic, memory access, branching).

4. Describe the role of the instruction register (IR) in control flow.

   The Instruction Register (IR) holds the current instruction fetched from memory. It provides the decoded instruction to the CPU for execution, enabling the processor to know which operation to perform next.

5. Explain how the CPU ensures sequential execution of instructions in normal control flow.

   The CPU uses the Program Counter (PC) to ensure sequential execution.

   - The PC points to the memory address of the next instruction.
   - After fetching an instruction, the PC increments automatically to point to the subsequent instruction.
   - Sequential execution continues unless an instruction modifies the PC (e.g., a jump or branch).

6. Write a simple C program that demonstrates normal control flow for adding two numbers. Provide its Y86-64 assembly equivalent.

Dr.Suresh

C Code:

```
#include <stdio.h>

int main() {
    int a = 5, b = 3;
    int sum = a + b;
    printf("Sum: %d\n", sum);
    return 0;
}
```

Y86-64 Assembly:

```
irmovq $5, %rdi         # Load 5 into %rdi (a)
irmovq $3, %rsi         # Load 3 into %rsi (b)
addq %rsi, %rdi         # Add %rsi to %rdi (sum)
rrmovq %rdi, %rax       # Move sum to %rax
irmovq $result_str, %rsi # Load format string address
call printf             # Print the result
halt

result_str:
    .string "Sum: %d\n"
```

7. Illustrate the fetch-decode-execute process with an example instruction (e.g., *addq %rbx, %rax*).
   - Fetch: The CPU fetches the *addq* instruction from memory into the IR.
   - Decode: The CPU decodes the opcode to determine that it is an addition operation and identifies the operands (*%rbx* and *%rax*).
   - Execute: The CPU adds the value in *%rbx* to *%rax* and stores the result in *%rax*.
8. Compare and contrast normal control flow and exception control flow.
   - **Normal Control Flow**:
     - Follows sequential execution of program logic.
     - Deterministic and uninterrupted by external events.
     - Example: Sequential instructions, loops.
   - **Exception Control Flow**:
     - Triggered by events like interrupts or traps.
     - Non-deterministic and disrupts sequential execution.
     - Example: Page faults, division by zero, interrupts.
9. Design a simple assembly program in Y86-64 to demonstrate normal control flow with sequential, conditional, and loop constructs.

   Assembly Code:

```
irmovq $0, %rax          # Initialize sum = 0
irmovq $10, %rcx         # Counter = 10
```

loop:
addq %rcx, %rax # sum += counter subq $1, %rcx # counter– jne loop # Repeat if counter != 0 halt

Dr.Sureshp

20.2 Questions and Answers on Exceptions in Control Flow

1. Define an exception in the context of control flow.

   An exception is an event that disrupts the normal flow of program execution. It can be caused by hardware (e.g., division by zero) or software (e.g., invalid memory access). Exceptions transfer control to an exception handler.

2. List the types of exceptions that can interrupt normal control flow.

   - Interrupts: Caused by external hardware (e.g., I/O devices, timers).
   - Traps: Intentional exceptions generated by the program (e.g., system calls).
   - Faults: Recoverable errors (e.g., page faults).
   - Aborts: Non-recoverable errors (e.g., hardware failure).

3. What is the role of the interrupt vector in exception handling?

   The interrupt vector is a table that stores the addresses of exception handlers. When an exception occurs, the CPU uses the interrupt vector to determine which handler to invoke, ensuring the correct response to the exception.

4. Describe the steps involved in handling an external interrupt.

   - Interrupt Request: The hardware signals an interrupt to the CPU.
   - Save State: The CPU saves the current program counter (PC) and processor state.
   - Interrupt Vector Lookup: The CPU retrieves the handler address from the interrupt vector.
   - Execute Handler: The control flow switches to the interrupt handler to address the event.
   - Restore State: The CPU restores the saved state and resumes program execution.

5. Why does a segmentation fault occur, and how does it affect control flow?

   A segmentation fault occurs when a program accesses memory that it is not allowed to. This disrupts normal control flow, causing the CPU to invoke the operating system's fault handler, which typically terminates the program or raises a signal (e.g., SIGSEGV).

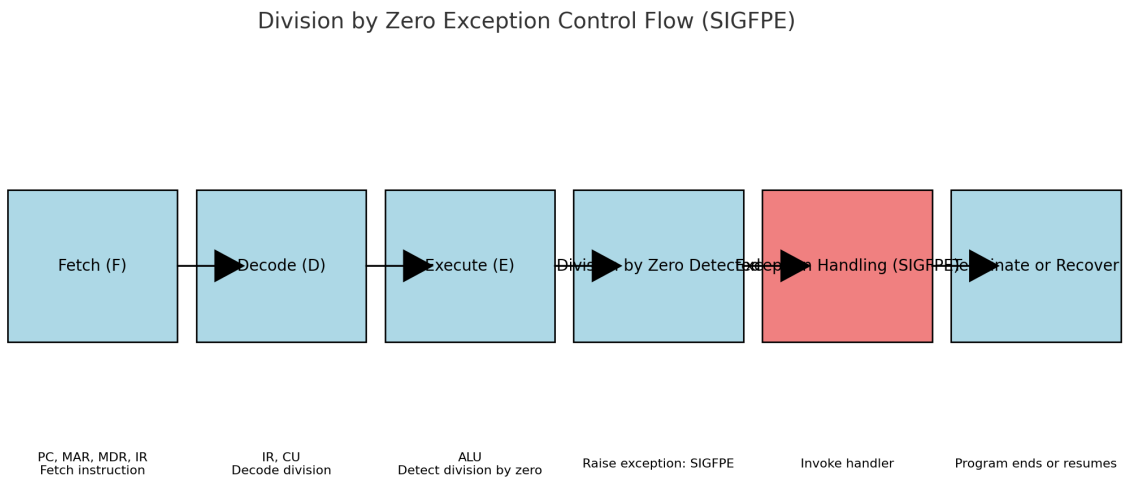
6. Illustrate the role of the operating system in handling a page fault exception.

   - Cause: A program accesses a memory page that is not in physical RAM.
   - Page Fault Trap: The CPU raises a page fault exception and transfers control to the OS.
   - Page Table Update:
     - The OS locates the requested page on disk.
     - Loads the page into a free RAM frame.
     - Updates the page table with the new mapping.
   - Resume Execution: The CPU retries the instruction causing the page fault, now with the updated mapping.

7. A division-by-zero exception occurs in a program. Outline the sequence of steps the CPU and OS take to handle this exception.

Dr.Sureshpareth

- The CPU detects a division-by-zero operation and triggers a floating-point exception (SIGFPE).
- The CPU saves the program counter and state.
- The OS looks up the appropriate handler in the interrupt vector table.
- Control is transferred to the SIGFPE handler.
- The handler:
  - Logs the error.
  - Attempts recovery or terminates the program.
- The CPU restores the saved state and resumes or exits.



8. Analyze how an exception alters the normal fetch-decode-execute cycle.
   - **Interrupt/Exception Detection**:
     - The CPU detects an exception (e.g., division by zero) during execution.
   - **Pipeline Flush**:
     - Ongoing instructions in the pipeline are discarded.
   - **Control Transfer**:
     - The CPU saves the current PC and processor state.
     - The control flow switches to the exception handler.
   - **Handler Execution**:
     - The handler resolves the exception or performs cleanup.
   - **Resumption**:
     - If recoverable, the CPU restores the program state and resumes execution.

21

Virtual Memory

21.1 Questions and Answers on Virtual Memory

1. Define virtual memory and explain its purpose. (2 marks)

   Virtual memory is a memory management technique that provides an abstraction of physical memory. It allows programs to use more memory than physically available by utilizing disk space as an extension of RAM. This provides isolation between processes and simplifies memory allocation.

2. List the key components of virtual memory. (3 marks)
   - **Page Table**: Maps virtual addresses to physical addresses.
   - **TLB (Translation Lookaside Buffer)**: A cache for page table entries to speed up address translation.
   - **Swap Space**: Disk storage used to temporarily hold data not currently in RAM.

3. Describe the advantages of virtual memory. (3 marks)
   - **Isolation**: Prevents one process from accessing another's memory.
   - **Efficient Use of RAM**: Only active pages are loaded into RAM.
   - **Supports Large Programs**: Allows programs to use memory beyond the physical limit of RAM.
   - **Simplified Memory Management**: Provides contiguous memory to processes.

4. Explain the concept of paging in virtual memory. (5 marks)

   Paging is a technique that divides virtual memory into fixed-size blocks called pages. These pages are mapped to physical memory blocks called frames. When a page is accessed, the page table determines its corresponding frame in RAM or triggers a page fault if not present in memory.

5. What is the difference between virtual address and physical address? (5 marks)
   - **Virtual Address**:
     - Logical address used by the CPU to access memory.
     - Generated by the program and mapped to physical memory by the OS.

Dr.Suresh Pare

- **Physical Address**:
  - Actual memory address in RAM.
  - Used by the hardware to fetch data.

6. Illustrate how virtual memory enables a 32 KB program to run on a system with 16 KB RAM.
   - The OS divides the program into pages (e.g., 4 KB each).
   - Active pages are loaded into RAM frames.
   - Inactive pages are stored in swap space on the disk.
   - Page replacement algorithms manage which pages are swapped in and out as needed.

Dr.Sureshpareth

22

Operating System

22.1 Introduction

An operating system acts as an intermediary between computer hardware and the user, managing resources efficiently and providing an environment for program execution.

Key Functionalities:

- **Resource Management**:
  - **CPU Management**: Allocates processor time using scheduling algorithms.
  - **Memory Management**: Manages memory allocation and deallocation efficiently.
  - **I/O Management**: Handles input/output device operations.
  - **Storage Management**: Organizes and retrieves files and directories.
- **Process Management**:
  - Handles creation, execution, and termination of processes.
  - Ensures inter-process communication and synchronization.
- **File System Management**:
  - Provides efficient storage and retrieval of data.
  - Ensures data access control and integrity.
- **Security and Protection**:
  - Implements authentication and access control mechanisms to protect data and resources.
- **Providing an Interface**:
  - Offers user-friendly interfaces such as CLI (Command-Line Interface) and GUI (Graphical User Interface).

Primary Objectives:

- Enhance user convenience.
- Maximize system performance and resource utilization.

Dr.Sureshpare

- Provide an environment for efficient program execution.

Dr.Sureshpareth

23

FUNDAMENTALS OF COMPUTER SYSTEMS

Sub Code Evaluation Criteria	IT111 QUIZ(2) + MIDTERM + ENDTERM
L-T-P	4-0-0
Total Hours Total Marks	45(T) 100
Credits Exam Hours	04 03

Dr.Suresh Kumar

24 Evaluation Criteria

Quiss(2) (20%);

Mid Term(30%);

Final Exam (50%).

Week	Content
Quiz	Covers Weeks 1-4
Midterm Examination	Covers Weeks 1-7
Endterm Examination	Covers Weeks 1-15

Dr.Suresh Pare

25

END SEMESTER EXAMINATION ANSWER KEY

Dr.Sureshpare

1. Suppose *a* and *b* are byte values with *a = 0x55* and *b = 0x45*, respectively. Evaluate the following expressions:

1. **(a)** *a & b*
2. **(b)** *a | b*
3. **(c)** *~a | ~b*
4. **(d)** *a && b*
5. **(e)** *!a || !b* [Marks 5*2]

Answer:

Given: - *a = 0x55* (hexadecimal) = *85* (decimal) = *01010101* (binary)

- *b = 0x45* (hexadecimal) = *69* (decimal) = *01000101* (binary)

(a) *a & b*:

This is a bitwise AND operation. Each bit in the result is *1* only if the corresponding bits in both *a* and *b* are *1*.

$$ab = 01010101$$
$$01000101 = 01000101$$

Result (in hexadecimal): *0x45* or *69* (decimal).

(b) *a | b*:

This is a bitwise OR operation. Each bit in the result is *1* if the corresponding bit in either *a* or *b* is *1*.

$$a|b = 01010101|01000101 = 01010101$$

Result (in hexadecimal): *0x55* or *85* (decimal).

(c) *~a | ~b*:

The tilde (~) is the bitwise NOT operator, which inverts every bit (1 becomes 0, and 0 becomes 1). Since *a* and *b* are bytes, they are 8 bits wide.

First, compute ~*a* and ~*b*:

$$a = \ 01010101 = 10101010$$
$$b = \ 01000101 = 10111010$$

Now, compute the OR operation:

$$a|\ b = 10101010|10111010 = 10111010$$

Result (in hexadecimal): *0xBA* or *186* (decimal).

(d) *a && b*:

This is a logical AND operation (not bitwise). It evaluates to *1* (true) if both *a* and *b* are non-zero, otherwise *0* (false).

Since both *a = 85* and *b = 69* are non-zero:

$$a$$
$$b = 1$$

Result: *1* (true).

(e) *!a || !b*:

The exclamation mark (*!*) is the logical NOT operator. It evaluates to *1* (true) if the operand is *0*, and *0* (false) otherwise.

- *!a = 0* (since *a != 0*)
- *!b = 0* (since *b != 0*)

Logical OR (||) results in *1* if either operand is *1*:

$$!a||!b = 0||0 = 0$$

Result: *0* (false).

1. **(a)** *a & b = 0x45* (decimal: *69*)
2. **(b)** *a | b = 0x55* (decimal: *85*)
3. **(c)** *~a | ~b = 0xBA* (decimal: *186*)
4. **(d)** *a && b = 1* (true)
5. **(e)** *!a || !b = 0* (false)

2.a) Explain the difference between *irmovq*, *rrmovq*, and *mrmovq* instructions in Y86-64.[Marks 5]

Answer:

1. *irmovq* (Immediate to Register Move):
   - Loads a constant value into a register.
   - **Example**: *irmovq $5, %rax* moves *5* into *%rax*.
   - **Machine Code**: *30 F0 05 00 00 00 00 00*.
2. *rrmovq* (Register to Register Move):
   - Moves data between registers.
   - **Example**: *rrmovq %rax, %rbx* copies *%rax* to *%rbx*.

Dr.Sureshpareth

- **Machine Code**: *20 03*.

3. *mrmovq* (Memory to Register Move):
   - Loads data from a memory address into a register.
   - **Example**: *mrmovq 8(%rbx), %rax* loads data from address *(%rbx + 8)* into *%rax*.
   - **Machine Code**: *50 30 08*.

2.b) Explain while running a Simple Program of subtracting two numbers by providing: C code, Assembly, Machine Code and CPU Role. [Marks 1 + 1 + 1 + 2]

Answer

The C program for subtracting two numbers:

```c
int main() {
    int a = 15;
    int b = 7;
    int result = a - b;
    return result;
}
```

The corresponding Y86-64 assembly code:

```
# Initialize values
irmovq $15, %rax        # Load 15 into %rax (a = 15)
irmovq $7, %rbx         # Load 7 into %rbx (b = 7)

# Perform subtraction
subq %rbx, %rax         # Compute a - b, store result in %rax (result = a - b)

# Return result
ret                     # Return from the function with result in %rax
```

The OS and CPU execute the program as follows:

1. **Compilation**:
   - The C compiler translates the code into Y86-64 assembly code and converts it into machine code.
   - Example machine code:

```
30F00F000000000000  # irmovq $15, %rax
30F307000000000000  # irmovq $7, %rbx
6130                # subq %rbx, %rax
90                  # ret
```

2. **Program Loading**:
   - The OS loads the machine code into memory.
   - Sets up:
     - **Code Section**: Contains machine code instructions.
     - **Data Section**: Allocates memory for variables.
     - **Stack Section**: Prepares a stack for function calls and returns.
3. **Process Initialization**:
   - The OS initializes the CPU:

  - Loads the Program Counter (PC) with the program's starting address (entry point).
  - Initializes general-purpose registers (*%rax*, *%rbx*, etc.).
  - Sets up the stack pointer (*%rsp*).

4. **Instruction Execution**:
   - The CPU executes instructions step by step:
     - `irmovq $15, %rax`: Loads the value *15* into *%rax*.
     - `irmovq $7, %rbx`: Loads the value *7* into *%rbx*.
     - `subq %rbx, %rax`: Subtracts the value in *%rbx* (*7*) from *%rax* (*15*) and stores the result (*8*) in *%rax*.
     - `ret`: Returns from the function.
5. **System Call to Return**:
   - The *ret* instruction signals the OS to handle the program's exit.
   - The OS retrieves the result from *%rax* and terminates the process.
6. **Cleanup**:
   - The OS deallocates memory and resets the CPU state for the next process.

The machine code for the assembly instructions is as follows:

Instruction	Opcode | Operands | Machine Code		
irmovq $15, %rax	0x30	Immediate = 15, %rax	30F00F000000000000
irmovq $7, %rbx	0x30	Immediate = 7, %rbx	30F307000000000000
subq %rbx, %rax	0x61	%rbx, %rax	6130
ret	0x90	N/A	90

Encoded Machine Code Sequence

The complete machine code sequence:

```
30F00F000000000000  # irmovq $15, %rax
30F107000000000000  # irmovq $7, %rbx
6123                # subq %rbx, %rax
90                  # ret
```

Output of the Program

- **Input**: *a = 15*, *b = 7*
- **Output**: *result = 8*, stored in *%rax*

The OS's Role:

1. **Program Loading**: Loads the binary into memory and initializes CPU registers.
2. **Process Execution**: Manages execution of instructions and CPU state.
3. **System Call Handling**: Processes the *ret* instruction and terminates the program.

3.a) Write C program and its corresponding assembly code for multiplying two numbers that are statically provided. [Marks 1 + 4]

Dr. Tirumaleshwaraprasad

Answer:

Multiplication of Two Numbers in Y86-64:

The C program for multiplying two integers:

```
int main() {
    int a = 4;
    int b = 6;
    int c = a * b;
    return c;
```

Since Y86-64 does not have a direct *mul* instruction, multiplication is implemented using repeated addition in a loop.

The assembly code for multiplying two numbers (a and b):

```
# Initialize values
irmovq $4, %rax         # Load 4 into %rax (a = 4)
irmovq $6, %rbx         # Load 6 into %rbx (b = 6)
irmovq $0, %rcx         # Initialize %rcx to 0 (c = 0)

# Multiply using repeated addition
andq %rbx, %rbx         # Check if %rbx (b) is zero
je done                 # If b == 0, jump to done
loop:
addq %rax, %rcx         # Add %rax (a) to %rcx (c += a)
subq $1, %rbx           # Decrement %rbx (b -= 1)
jne loop                # If b != 0, repeat loop

done:
rrmovq %rcx, %rax       # Move result (c) into %rax for return
ret                     # Return from the function
```

The corresponding machine code for the assembly instructions is shown below:

Instruction	Opcode | Operands | Machine Code		
irmovq $4, %rax	0x30	Immediate = 4, %rax	30F004000000000000
irmovq $6, %rbx	0x30	Immediate = 6, %rbx	30F306000000000000
irmovq $0, %rcx	0x30	Immediate = 0, %rcx	30F100000000000000
andq %rbx, %rbx	0x62	%rbx, %rbx	6233
je done	0x71	Offset = 17	7117
addq %rax, %rcx	0x60	%rax, %rcx	6001
subq $1, %rbx	0x61	%rbx, Immediate = 1	61F301000000000000
jne loop	0x72	Offset = 12	7212
rrmovq %rcx, %rax	0x20	%rcx, %rax	2010
ret	0x90	N/A	90

3.b) Write a C program and Y86-64 assembly program to compute the sum of numbers from 1 to 5 using the for loop. [Marks 2 + 3]

Answer:

C Code:

```
#include <stdio.h>
```

(continues on next page)

(continued from previous page)

```c
int main() {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {
        sum += i;
    }
    printf("Sum: %d\n", sum);
    return 0;
}
```

Y86-64 Assembly Code:

```
irmovq $0, %rax          # Initialize sum to 0 (sum = 0)
irmovq $1, %rbx          # Initialize loop counter to 1 (i = 1)
irmovq $5, %rcx          # Upper limit for loop (i <= 5)

loop:
    addq %rbx, %rax      # Add i to sum (sum += i)
    addq $1, %rbx        # Increment loop counter (i++)
    subq %rbx, %rcx      # Check if i <= 5
    jne loop              # If i <= 5, jump back to loop

irmovq $result_str, %rdi # Load address of format string
rrmovq %rax, %rsi        # Move sum to %rsi (second argument for printf)
call printf              # Call printf to print sum
halt                     # End program

result_str:
    .string "Sum: %d\n"  # Format string for printf
```

4.a) Explain pipeline, pipeline hazard and solution to handling hazard. [Marks 1 + 2 + 2]

Answer:

Pipelining increases processor throughput by overlapping the execution of multiple instructions. Each instruction progresses through different stages (Fetch, Decode, etc.), so the CPU always works on several instructions simultaneously.

Pipeline stages divide instruction execution into smaller steps:

Pipelining divides the process of executing an instruction into smaller steps called pipeline stages. Each stage handles a specific part of the instruction's execution. By overlapping these stages for multiple instructions, pipelining increases the throughput of the processor.

Pipeline Stages in Y86-64:

Fetch (F):

The instruction is fetched from memory at the address specified by the Program Counter (PC).

The fetched instruction is stored in the Instruction Register.

Example: Fetch irmovq $5, %rax from memory at PC = 0x10.

Decode (D):

The instruction is decoded to determine its type and operands, including registers if required.

Example: Decode *irmovq* to identify *$5* as an immediate value and *%rax* as the destination register.

Execute (E):

Dr.Suresh

The ALU performs the specified operation using the operands provided during the Decode stage.

Example: In *addq %rbx, %rax*, the ALU adds the values in *%rbx* and *%rax*.

Memory Access (M):

If the instruction involves memory (e.g., *mrmovq* or *rmmovq*), this stage reads from or writes to memory.

Example: For *mrmovq 8(%rbx), %rax*, data is fetched from memory address *(%rbx + 8)* and stored in *%rax*.

Write-back (W):

The result of the computation or memory operation is written back to the destination register or memory.

Example: In *irmovq $5, %rax*, the value *5* is written to *%rax*.

Role of Pipeline Stages in Instruction Execution

Each stage focuses on a specific sub-task, allowing multiple instructions to progress through different stages simultaneously.

Example Execution:

- Cycle 1: Fetch *irmovq $5, %rax*.
- Cycle 2: Decode *irmovq $5, %rax* while fetching the next instruction.
- Cycle 3: Execute *irmovq $5, %rax* while decoding the next instruction and fetching another.

Data Hazards:

Occur when one instruction depends on the result of a previous instruction still in progress.

Example: Using *%rax* in *subq* before it is updated in *addq*.

Control Hazards:

Occur during branch instructions (*jmp*, *jg*) when the next instruction isn't known until the branch condition is resolved.

Structural Hazards:

Occur when two instructions require the same hardware resource simultaneously.

Pipelining stages like Fetch, Decode, Execute, Memory Access, and Write-back work together to maximize processor efficiency. Y86-64's simple architecture demonstrates these concepts effectively, preparing students to understand more complex real-world processors like x86-64.

- **Data Hazards**:

  Occur when an instruction depends on the result of a previous one.

  Example:

  ```
  addq %rbx, %rax
  subq %rax, %rcx
  ```

  Resolution: Forwarding or stalling.

- **Control Hazards**:

  Occur during branch instructions.

Dr.Sureshpareth

Example:

```
jg greater
```

Resolution: Branch prediction or stalling.

The role of forwarding in resolving control hazards in a pipeline:

Forwarding, also known as bypassing, resolves data hazards by:

- Sending the output of the ALU directly to the next instruction's Execute stage.
- Eliminating the need to wait for Write-back.

Example:

```
addq %rbx, %rax   # ALU computes %rax = %rax + %rbx
subq %rax, %rcx   # Uses ALU output directly, bypassing Write-back
```

The ALU result from *addq* is forwarded to the input of *subq*, allowing execution without a stall.

The role of stalls in resolving control hazards in a pipeline:

A control hazard occurs during a branch instruction when the processor doesn't know the next instruction to fetch. The role of stalls is to pause instruction fetching until the branch outcome is known.

Example:

```
subq %rsi, %rdi    # Compare
jg greater          # Conditional jump
```

The pipeline cannot fetch the next instruction until the branch condition (*jg*) is resolved. Stalling pauses the Fetch stage for subsequent instructions until the branch target is computed.

4.b) Analyze a pipelined execution of the following Y86-64 code: [Marks 5]

Code:

```
irmovq $5, %rax
irmovq $3, %rbx
addq %rbx, %rax
jmp done
done:
halt
```

Answer:

The pipelined execution processes multiple instructions simultaneously across different pipeline stages (Fetch, Decode, Execute, Memory, Write-back).

Fetch:

Fetch *irmovq $5, %rax*.

Fetch *irmovq $3, %rbx* while decoding the first instruction.

- **Decode**:

  Decode *irmovq $3, %rbx* while executing the first instruction.

Dr. Suresh Parsath

- **Execute**:

  Execute *addq %rbx, %rax* after decoding *jmp done*.

Hazards:

- **Data Hazard**:

  addq %rbx, %rax depends on *%rbx* being updated.

  Resolution: Use forwarding to pass the value of *%rbx* directly to the ALU.

- **Control Hazard**:

  jmp done creates uncertainty about the next instruction.

  Resolution: Use a stall to wait until the jump target is resolved.

Pipeline Stages:

Cycle	Fetch	Decode	Execute	Memory	Write-back
1	*irmovq $5, %rax*				
2	*irmovq $3, %rbx*	*irmovq $5, %rax*			
3	*addq %rbx, %rax*	*irmovq $3, %rbx*	*irmovq $5, %rax*		
4	*jmp done*	*addq %rbx, %rax*	*irmovq $3, %rbx*	*irmovq $5, %rax*	
5	*halt*	*jmp done*	*addq %rbx, %rax*	*irmovq $3, %rbx*	*irmovq $5, %rax*
6		*halt*	*jmp done*	*addq %rbx, %rax*	*irmovq $3, %rbx*

Potential Hazards:

- **Data Hazard**:

  The *addq %rbx, %rax* instruction depends on *%rbx* being updated.

  Resolution: Forward *%rbx* directly from Decode or Execute to the ALU.

- **Control Hazard**:

  The *jmp done* instruction creates uncertainty about the next instruction.

  Resolution: Stall until the target address is resolved.

Machine Code:

Dr.Suresh

Address	Instruction	Machine Code	Explanation
0x0000	*irmovq $0, %rax*	*30 F0 00 00 00 00 00 00 00*	Initialize sum to 0.
0x000A	*irmovq $1, %rbx*	*30 F3 01 00 00 00 00 00 00*	Initialize counter to 1.
0x0014	*irmovq $5, %rcx*	*30 F1 05 00 00 00 00 00 00*	Set upper limit to 5.
0x001E	*addq %rbx, %rax*	*60 30*	Add %rbx to %rax (sum += i).
0x0020	*addq $1, %rbx*	*30 F3 01 00 00 00 00 00 00*	Increment %rbx (i++).
0x002A	*subq %rcx, %rbx*	*61 13*	Compare %rbx (i) with %rcx (5).
0x002C	*jl loop*	*70 XX XX XX XX*	Jump to *loop* if i <= 5.
0x0030	*irmovq $result_str, %rdi*	*30 F7 XX XX XX XX XX XX XX*	Load address of format string.
0x003A	*rrmovq %rax, %rsi*	*20 06*	Move sum to %rsi.
0x003C	*call printf*	*80 XX XX XX XX XX XX XX*	Call printf to print sum.
0x0044	*halt*	*00*	End program.
0x0050	*result_str*	ASCII for “Sum: %dn”.	Format string.

Explanation:

1. **Initialization**:
   - *%rax* stores the *sum*, initialized to 0.
   - *%rbx* stores the loop counter (*i*), initialized to 1.
   - *%rcx* stores the upper limit (5).
2. **Loop Execution**:
   - The instruction *addq %rbx, %rax* adds the current value of *%rbx* (i.e., *i*) to *%rax* (sum).
   - *%rbx* is incremented with *addq $1, %rbx* in each iteration.
   - The condition *subq %rcx, %rbx* checks if *%rbx* exceeds the upper limit (5).
3. **Condition Check**:
   - If *%rbx* (i) is less than or equal to *%rcx* (5), the program jumps back to the *loop* label (*jl loop*).
4. **Final Output**
   - After exiting the loop, the value of *sum* is moved to *%rsi*, which is the second argument for *printf*.
   - The format string is loaded into *%rdi*.
   - *printf* is called to print the result.

5.Write a program that includes a function add(int a, int b) defined in the same source file and called from main(). Explain the process of static linking when the program is compiled and executed. Illustrate the memory layout of the executable file after linking. [Marks 2 + 4 + 4]

Dr.Suresh

Answer:

C Code:

```c
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(5, 3);
    printf("Result: %d\n", result);
    return 0;
}
```

Assembly Code:

```
add:
    addq %rbx, %rax          # %rax = %rax + %rbx
    ret                      # Return to caller

main:
    irmovq $5, %rax          # Load 5 into %rax (argument a)
    irmovq $3, %rbx          # Load 3 into %rbx (argument b)
    call add                 # Call add(a, b)
    rrmovq %rax, %rcx        # Move result from %rax to %rcx
    irmovq $result_str, %rdi # Load address of format string into %rdi
    call printf              # Call printf to print result
    ret                      # Return from main

result_str:
    .string "Result: %d\n"   # String format for printf
```

Object File: Machine Code:

Address	Instruction	Machine Code
0x00	addq %rbx, %rax	60 30
0x02	ret	90
0x10	irmovq $5, %rax	30 F0 05 00 00 00 00 00 00
0x1A	irmovq $3, %rbx	30 F3 03 00 00 00 00 00 00
0x24	call add	80 00 00 00 00 00 00 00 00
0x2C	rrmovq %rax, %rcx	20 02
0x2E	irmovq $result_str, %rdi	30 F7 XX XX XX XX XX XX XX
0x38	call printf	80 XX XX XX XX XX XX XX XX
0x40	ret	90

Symbol Table:

Symbol Name	Type	Address	Defined/Referenced	Description
add	Function	0x00	Defined	Start of the add function.
main	Function	0x10	Defined	Start of the main function.
printf	External	None	Referenced	Standard library function.
result_str	Data	XX	Defined	Format string for printf.

Static Linking Explanation:

Since both *add* and *main* are in the same source file, the compiler and assembler produce a single object file (*main.o*). Here is how static linking is applied:

1. **Symbol Resolution**:
   - The *call add* instruction in the *main* function is resolved to the address *0x00*, where the *add* function starts.
2. **No Relocation Required**:
   - Since the *add* and *main* functions are within the same file, their addresses are resolved at compile time, and no relocation is needed for these symbols.
3. **External Symbols**:
   - The *printf* function is external, so its address is resolved by linking the object file with the standard library during the linking phase.
4. **Final Executable**:
   - The executable contains machine code for both *add* and *main*, with all symbols and memory addresses fully resolved.

Final Memory Layout After Static Linking:

Address	Instruction / Data	Machine Code	Explanation
0x0000	addq %rbx, %rax	60 30	*add* function starts: add two numbers.
0x0002	ret	90	Return from *add*.
0x0010	irmovq $5, %rax	30 F0 05 00 00 00 00 00 00	Load *5* into *%rax* (argument to *add*).
0x001A	irmovq $3, %rbx	30 F3 03 00 00 00 00 00 00	Load *3* into *%rbx* (argument to *add*).
0x0024	call add	80 00 00 00 00 00 00 00 00	Call *add* function at *0x0000*.
0x002C	rrmovq %rax, %rcx	20 02	Move result from *%rax* to *%rcx*.
0x002E	irmovq $result_str, %rdi	30 F7 50 00 00 00 00 00 00	Load address of format string (*result_str*) into *%rdi*.
0x0038	call printf	80 80 00 40 00 00 00 00 00	Call *printf* at *0x400080*.
0x0040	ret	90	Return from *main*.
0x0050	.string "Result: %dn"	ASCII encoding	Format string for *printf*.
0x40008	printf implementation	Binary machine code	Statically linked *printf* function from *libc.a*.

Based on the scenario given below, answer the questions 6 and 7 (b):

```
#include <stdio.h>
int main() {
int arr[10000];
int sum = 0;
for (int i = 0; i < 10000; i++) {
sum += arr[i];
```

(continues on next page)

(continued from previous page)

```
}
return 0;
}
```

Given the following details:

- L1 Cache Size: 64 KB
- Cache Line Size: 64 bytes
- Array Size: 10,000 integers (each of size 4 bytes)

6.a). Calculate:

- The number of elements that can fit in a single cache line. [Marks 2]
- The total number of cache lines required to store the entire array. [Marks 2]

b). Determine if the L1 cache can hold the entire array. [Mark 1]

c). Assuming the array is accessed sequentially, compute the number of cache misses and cache hits. [Marks 3 + 2]

Answer:

Given Details:

- L1 Cache Size: 64 KB = 64 × 1024 = 65,536 bytes.
- Cache Line Size: 64 bytes.
- Array Size: 10,000 × 4 = 40,000 bytes (since each integer is 4 bytes).

a1: Number of Elements per Cache Line

Each cache line can hold:

$$\text{Cache Line Capacity} = \frac{\text{Cache Line Size}}{\text{Element Size}} = \frac{64}{4} = 16\,\text{elements.}$$

a2: Number of Cache Lines Required for the Array

$$\text{Total Cache Lines Required} = \frac{\text{Array Size}}{\text{Cache Line Size}} = \frac{40000}{64} = 625\,\text{cache lines.}$$

b: Can L1 Hold the Entire Array?

Number of cache lines L1 can hold:

$$\text{L1 Cache Capacity in Cache Lines} = \frac{\text{L1 Cache Size}}{\text{Cache Line Size}} = \frac{65536}{64} = 1024\,\text{cache lines.}$$

Since 625 < 1024, L1 can hold the entire array.

c: Cache Hits and Misses

Assumptions:

- Array arr[10000] is accessed sequentially.
- Cache is initially empty.

First Access (arr[0]):

- A miss occurs as the data is not in L1.
- The first cache line (containing arr[0] to arr[15]) is loaded.

Dr.Sureshpareth

Subsequent Accesses:

- Accessing arr[1] to arr[15] results in hits because they are in the same cache line.
- Accessing arr[16] triggers a miss, loading a new cache line (containing arr[16] to arr[31]).

So,

- One miss per cache line.
- For 625 cache lines, there are 625 misses.

Total Array Accesses:

- Total accesses: 10,000.
- Hits: $10{,}000 - 625 = 9{,}375$.

Hence

- Number of Elements per Cache Line: 16 elements.
- Number of Cache Lines Required for Array: 625.
- L1 Can Hold the Entire Array: Yes.
- Number of Cache Misses: 625.
- Number of Cache Hits: 9,375.

7.a) Explain the Principle of Locality. [Marks 5]

Answer:

Principle of Locality:

The principle of locality is a fundamental concept in computer science, particularly in the design of computer memory systems and performance optimization. It describes how programs tend to access a small portion of data or instructions repeatedly over a short period.

This principle can be divided into two types:

1. Temporal Locality

- **Definition**: If a memory location is accessed at a given time, it is likely to be accessed again in the near future.
- **Example**:
  - When a program accesses a variable in a loop, it is likely to access the same variable repeatedly during the loop's execution.

2. Spatial Locality

- **Definition**: If a memory location is accessed, nearby memory locations are likely to be accessed soon.
- **Example**:
  - When traversing an array, accessing one element often leads to accessing adjacent elements.
  - Similarly, instructions stored sequentially in memory (e.g., in a function) are often executed sequentially.

Dr.Sureshpareth

7.b) Identify variables exhibiting spatial locality and temporal locality with respect to the code given above. [Marks 2 + 3]

Answer:

Identifying Locality of Reference:

Spatial Locality:

The array arr exhibits spatial locality. As the summation loop accesses consecutive elements of arr[i], the same cache line is reused for 16 consecutive accesses.

Temporal Locality:

The variable sum exhibits temporal locality. Its value is reused in every iteration of the loop as it accumulates the sum.

8.Explain the normal control flow during program execution with a corresponding diagram, highlighting the role of each CPU component in the fetch cycle. [Marks 10]

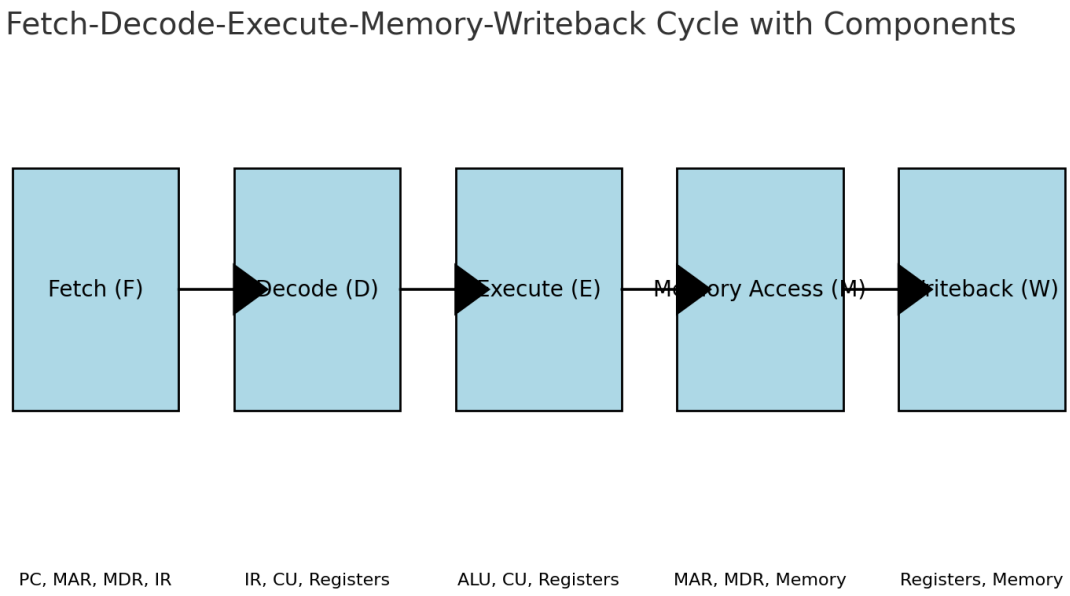
Answer:

> Normal control flow refers to the sequential execution of instructions in the order they are written in a program, starting from the program entry point (e.g., *main* in C), without interruptions like exceptions or external events.
>
> Example of F-D-E-M-W Cycle
>
> **Instruction:** `addq %rbx, %rax`
>
> This Y86-64 instruction adds the value in `%rbx` to `%rax`.

Cycle Breakdown:

- **Fetch:**
  - The instruction `addq %rbx, %rax` is fetched from memory.
- **Decode:**
  - The Control Unit decodes the opcode `addq`.
  - Operands `%rbx` (source) and `%rax` (destination) are identified.
- **Execute:**
  - The ALU adds the values in `%rbx` and `%rax`.
- **Memory Access:**
  - Not applicable for this instruction since no memory is directly involved.
- **Writeback:**
  - The result is written back to `%rax`.

Dr.Sureshpareth



A simple assembly program in Y86-64 to demonstrate normal control flow with sequential, conditional, and loop constructs.

Assembly Code:

```
irmovq $0, %rax          # Initialize sum = 0
irmovq $10, %rcx         # Counter = 10
```

loop:

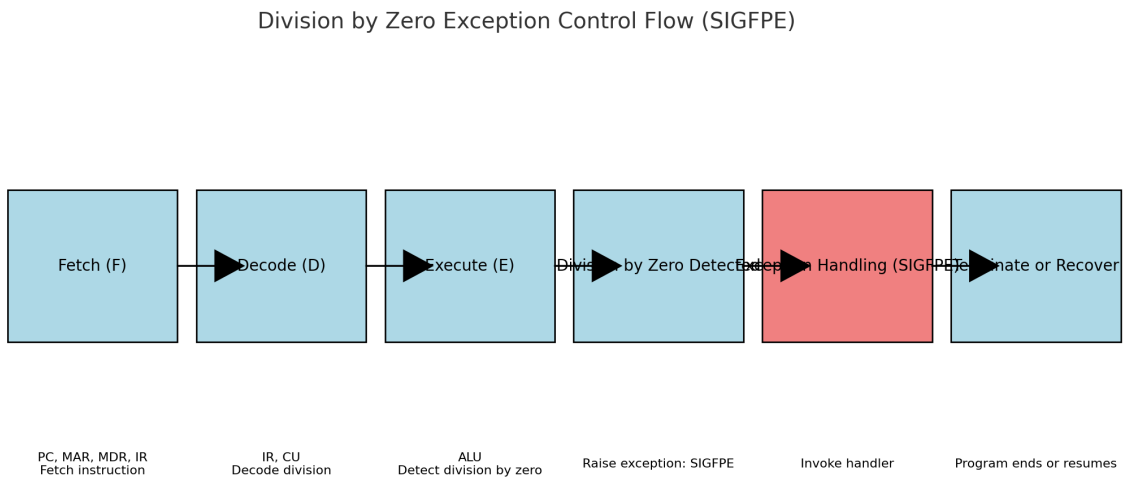
addq %rcx, %rax # sum += counter subq $1, %rcx # counter– jne loop # Repeat if counter != 0 halt

9.A division-by-zero exception occurs in a program. Outline the sequence of steps the CPU and OS take to handle this exception.[Marks 10]

Answer:

- The CPU detects a division-by-zero operation and triggers a floating-point exception (SIGFPE).
- The CPU saves the program counter and state.
- The OS looks up the appropriate handler in the interrupt vector table.
- Control is transferred to the SIGFPE handler.
- The handler:
  - Logs the error.
  - Attempts recovery or terminates the program.
- The CPU restores the saved state and resumes or exits.

Dr.Suresh



10.a) What is the difference between virtual address and physical address? [Marks 5]

Answer:

- **Virtual Address**:
  - Logical address used by the CPU to access memory.
  - Generated by the program and mapped to physical memory by the OS.
- **Physical Address**:
  - Actual memory address in RAM.
  - Used by the hardware to fetch data.

10.b) Illustrate how virtual memory enables a 32 KB program to run on a system with 16 KB RAM. [Marks 5]

Answer:

- The OS divides the program into pages (e.g., 4 KB each).
- Active pages are loaded into RAM frames.
- Inactive pages are stored in swap space on the disk.
- Page replacement algorithms manage which pages are swapped in and out as needed.

Dr.Sureshp